

v7 Model Release

September 3, 2025

Plain-English Summary

1. **Adjusted Performance got smarter about opponents.** I no longer judge a fighter against a generic average or the opponent's last fight. I build what that opponent typically allows and blend it with the division norm. If the opponent is historically stingy, landing on them counts more.
2. **I finally separated counts from binaries.** Counts (strikes landed/attempted, takedowns, reversals, etc.) are stabilized as rates per minute. Binary outcomes (KO, win, decision, sub landed) and control share get probability-style smoothing. Different problems, different tools.
3. **The pipeline order prevents "peeking."** I smooth first, keep a temporary `*_raw`, compute derived stats on the smoothed values, then drop the raw columns. Opponent features and weight-class priors are in place before adjusted performance is computed.
4. **Small samples don't hijack scores.** When the opponent has little history, I lean more on the weight class. As real data accumulates, it naturally takes over. Extreme one-off nights are clipped so they don't dominate training.

Deep Technical Analysis (How to Reproduce It)

0) Pipeline Order (high level)

1. Create base fight stats and copy to a derived table (so the rest of the pipeline has a stable surface).
2. **Beta-Binomial smoothing** for binary families (KO, win, decision, sub landed, control share). Run this first so attempts like `sub_att` are still raw.
3. **Poisson-Gamma smoothing** for count families (e.g., `sig_str_land`, `td_land`, `sub_att`, `kd`, `rev`).
4. Rename: temporarily keep originals as `*_raw`, replace originals with smoothed values, compute totals/accuracy/defense/ratios/per, then **delete** `*_raw`.
5. Create feature-specific tables; compute "per" features; build opponent features (what others did against this fighter).
6. Compute weight-class means, MAD, and a per-stat minimum MAD floor.
7. **Adjusted Performance** (non-decayed then decayed) on the feature-specific tables.

1) Adjusted Performance

Old behavior (why it was wrong)

- Baseline came from the opponent's *last fight row*: $(A.stat - B.prev_opp_avg) / B.prev_opp_mad$. One quirky row could swing the score.
- First-fight opponents fell straight to a weight-class table. No reliability-based blending.
- Denominator was a single-row MAD with a hard floor. That prevents division explosions but doesn't capture uncertainty.

New behavior (what to implement)

1. **Opponent history per feature table.** For a target column `c` (e.g., `head_acc`), gather all past fights where other fighters faced the current opponent. Compute:
 - `opp_mean_pers(c)`: mean of what others achieved against this opponent. If time-decay is on, weight each row `w = exp(-λ · years)` and use weighted mean.

- `opp_mad_pers(c)` : MAD via a two-step procedure: median of `val` , then median of `|val - median|` .
- `n` : sample size. With decay, use Kish effective `n`: $n = (\sum w)^2 / \sum w^2$.

```
-- skeleton of the per-column CTEs
rows_c (fight_id, fighter_id, val, w)
med_c (median per fight_id, fighter_id)
rows_with_m (join rows_c to med_c)
stats_c (mean val: SUM(w*val)/SUM(w) if decay, else AVG(val))
mad_c (median |val - median|)
```

2. Weight-class priors. Precompute for every feature table and column:

- `c_wc_mean` , `c_wc_mad` , and `c_mad_floor` .

```
-- each table: <table>_wc_mean, <table>_wc_mad, <table>_minimum_
```

3. Reliability-weighted shrinkage.

```
w_mean = n / (n + K_mean)
w_mad = n / (n + K_mad)

mu = w_mean * opp_mean_pers + (1 - w_mean) * wc_mean
sigma = max( w_mad * opp_mad_pers + (1 - w_mad) * wc_mad, mad_flo

adjperf = clip( (observed - mu) / sigma , -7, 7 )
```

- Defaults used: `K_mean = 4.0` , `K_mad = 4.0` (tune per family if you want; often `K_mad > K_mean` is safer).
- Observed is the already-smoothed feature value from the feature-specific table.

Data hygiene you must enforce

- Only use fights strictly before the current event; tie-break same-day events with `(event_id, fight_id)` .
- If a weight class lacks priors, fall back to a global row (don't coalesce to zero).
- Recommended hardening: compute a *per-column* effective `n` and set the weight to zero if that column has no history (instead of coalescing

personal stats to 0).

2) Poisson-Gamma Smoothing (Counts)

Scope

Columns ending with `_land` or `_att`, plus `kd` and `rev` (and their `_rd1` forms). Exclude static fields and binary/duration families.

Exposure definitions

- `_rd1` columns: `t = min(time_sec_rd1, 300) / 60.0`
- All others: `t = time_sec / 60.0`

Priors as rates (division-level, with global fallback)

```
wc_rate(c)      = SUM(c) / NULLIF(SUM(t), 0)
global_rate(c) = SUM(c) / NULLIF(SUM(t), 0)
```

Posterior and smoothed count

```
λ_post  = (wc_rate * τ + X) / (τ + t)
X_smooth = t * λ_post
```

Pseudo-minutes τ (global defaults used)

- Striking: `sig_str = 0.7`, `head = 0.8`, `body = 2.5`, `leg = 2.1`
- Grappling: `td = 7.0`, `sub = 12.0`, `kd = 20.0`, `rev = 42.0`
- Round 1: `sig_str_rd1 = 0.7`, `head_rd1 = 0.7`, `body_rd1 = 2.5`,
`leg_rd1 = 1.7`, `td_rd1 = 9.0`, `sub_rd1 = 15.0`, `kd_rd1 = 12.0`,
`rev_rd1 = 60.0`

Per-weight-class overrides (only where cross-validation helped)

- Flyweight: `rev = 22.0` (vs 42.0 global)
- Light Heavyweight: `head_rd1 = 0.5` (vs 0.7)
- Heavyweight: `td = 5.0` (vs 7.0), `td_rd1 = 4.0` (vs 9.0)

Execution notes

- Emit `c_smooth` values with a `_smooth` suffix; then rename so smoothed values replace originals. Keep `c_raw` only long enough to compute

totals/accuracy/defense/ratios/per; then drop it.

- Filter prior CTEs to rows with `time_sec > 0`.

3) Beta-Binomial Smoothing (Binary + Control Share)

Families and attempts

- `ko`, `win`, `decision`: successes = 0/1; attempts = 1 per fight.
- `sub_land`: successes = `sub_land`; attempts = `sub_att`.
- `ctrl`, `ctrl_rd1` (duration as share): attempts = seconds
(`min(rd1, 300)` for `rd1`); we output smoothed seconds = `p_post × attempts`.

Posterior mean

```
p_post = (rate_prior * τ + successes) / (τ + attempts)
if attempts = 0: p_post = rate_prior
```

Pseudo-counts τ (global defaults used)

- Global: `ko = 23`, `win = 25`, `decision = 20`, `sub_land = 9`, `ctrl = 2`
- Round 1: `ko_rd1 = 17`, `win_rd1 = 15`, `decision_rd1 = 16`,
`sub_land_rd1 = 7`, `ctrl_rd1 = 1`

Per-weight-class overrides (data-validated)

- Featherweight: `sub_land = 3` (vs 9)
- Light Heavyweight and Heavyweight: `ctrl = 1.5` (vs 2.0)

Priors

```
wc_rate = SUM(successes) / NULLIF(SUM(attempts), 0)
global_rate as fallback
```

Execution notes

- Run Beta-Binomial before Poisson-Gamma so attempts like `sub_att` are unmodified when you need them.
- Store `*_smooth`; then participate in the same rename step as counts.

4) Opponent Features and Priors (plumbing that makes AdjPerf work)

1. **Opponent "allowed" stats:** compute what other fighters did against each fighter (this is the "personal history" the opponent carries into a matchup).
2. **Weight-class aggregates:** build per-feature `_wc_mean`, `_wc_mad`, and `_minimum_mad` tables. These are used as priors and floors in AdjPerf.
3. **Strict time ordering:** when joining history to a current fight, only include rows strictly earlier than the current fight's event date (with tie-breakers).

5) Minimal SQL/Pseudocode you can adapt

Opponent history for a column c (decay optional)

```
rows_c AS (
SELECT cur.fight_id, cur.fighter_id, hist_opp.c AS val,
CASE WHEN :decay THEN EXP(-lambda * age_years) ELSE 1.0 END AS w
FROM features.<table> cur
JOIN fight_mapping fm_cur ON cur.fight_id = fm_cur.fight_id
JOIN event_mapping em_cur ON fm_cur.event_id = em_cur.event_id
-- figure out opponent id for the current fight
-- join to all past fights where others faced that opponent
-- restrict to strictly earlier fights (event_date, event_id, fight_id)
)
med_c AS (
SELECT fight_id, fighter_id,
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY val) AS med
FROM rows_c WHERE val IS NOT NULL GROUP BY fight_id, fighter_id
)
stats_c AS (
SELECT fight_id, fighter_id,
SUM(w*val) / NULLIF(SUM(CASE WHEN val IS NOT NULL THEN w END),0) AS c_op
FROM rows_c GROUP BY fight_id, fighter_id
)
mad_c AS (
SELECT fight_id, fighter_id,
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY ABS(val - med)) AS c_opp_mad
FROM rows_c JOIN med_c USING(fight_id, fighter_id)
GROUP BY fight_id, fighter_id
)
```

```
n_hist AS (
SELECT fight_id, fighter_id,
CASE WHEN :decay THEN POWER(SUM(w),2)/NULLIF(SUM(POWER(w,2)),0) ELSE COU
FROM rows_c GROUP BY fight_id, fighter_id
)
```

Adjusted Performance scoring

```
-- inputs: observed c, n, c_opp_mean_pers, c_opp_mad_pers, c_wc_mean, c_
w_mean = n / (n + K_mean)
w_mad = n / (n + K_mad)
mu = w_mean * c_opp_mean_pers + (1 - w_mean) * c_wc_mean
sigma = GREATEST(w_mad * c_opp_mad_pers + (1 - w_mad) * c_wc_mad, c_mad_
score = GREATEST(LEAST((observed - mu) / sigma, 7.0), -7.0)
```

Count smoothing (Poisson-Gamma)

```
t          = CASE WHEN is_rd1 THEN LEAST(time_sec_rd1,300)/60.0 ELSE time_
wc_rate = SUM(c)/SUM(t) (per weight class; have a global fallback)
X_smooth = t * ((wc_rate *  $\tau$  + X) / ( $\tau$  + t))
```

Binary smoothing (Beta-Binomial)

```
n = attempts -- the number of attempts for that row
x = successes -- the observed successes for that row
rate_p = wc_rate or global_rate if wc missing
p_post = (rate_p *  $\tau$  + x) / ( $\tau$  + n)
output = p_post -- or p_post * n for control seconds
```

6) Practical QA checks

- Distribution sanity: after smoothing, per-minute rates and binary probabilities should cluster around WC priors for tiny n and drift toward personal rates as n grows.
- Leakage audit: for a random sample of fights, verify that no future rows contribute to opponent-history or priors.
- Sensitivity: n=0,1,2 cases should produce conservative AdjPerf magnitudes; heavy shrink on dispersion reduces fake "z-score" spikes.

7) The Reality Check: Performance vs. Expectations

After all this technical work—the opponent-aware adjusted performance, the Poisson-Gamma smoothing, the Beta-Binomial calibration—you might expect dramatic improvements in accuracy or log-loss. The reality is more sobering.

We're still hitting roughly **70% accuracy on the unseen test set** (2024-2025 fights), with log-loss numbers virtually identical to previous model versions. The fundamental limitation isn't in our statistical methodology—it's in our feature set.

What Actually Improved

The main advantage of v7 is **better calibration**. The model's confidence scores now align more closely with actual win frequencies. This means:

- When the model is wrong, it tends to be *less confident* about those incorrect picks
- We can be more selective about +EV betting opportunities, leading to slightly better ROI
- The calibration curve shows the model is less overconfident in the 60-80% probability range

The Feature Ceiling Problem

We're fundamentally limited by using the same base statistics that every MMA model relies on: strikes landed/attempted, takedowns, control time, etc. These are the stats that have been available consistently across the last 10+ years of UFC data, but they only capture so much of what determines fight outcomes.

Real improvements in accuracy would require fundamentally new features:

- **Sentiment analysis** from fighter interviews, social media, and press conferences
- **Meta-analysis** of other successful handicappers and their picking patterns
- **Contextual factors** like training camp disruptions, weight cuts, personal circumstances
- **Video analysis** of technique, footwork, and tactical tendencies

The problem is consistency—these features are either unavailable for historical fights or extremely difficult to collect reliably across thousands of matchups.

The LLM Temptation (and Its Limitations)

Large Language Models offer an intriguing possibility for incorporating qualitative analysis, but they present a fundamental backtesting problem. You can't tell an LLM to "forget everything that happened after 2022" to create a proper train/test split. Without the ability to backtest on truly unseen data, it's impossible to know if LLM-enhanced predictions would actually generalize or just memorize recent fight outcomes.

This creates a catch-22: the most promising avenues for improvement (LLMs, real-time sentiment, insider information) are exactly the ones that break our ability to validate models scientifically.

Moving Forward

v7 represents the practical ceiling for what's achievable with traditional fight statistics and rigorous backtesting methodology. The technical improvements here—better smoothing, opponent-aware features, proper calibration—are the difference between a good model and a great one, even if that difference is measured in percentage points rather than dramatic leaps.

Future breakthroughs will likely come from entirely new data sources rather than more sophisticated processing of the same old numbers. Until then, we'll continue refining the edges and squeezing every bit of signal from the statistics we have.

Data Drift, Generalization, and the Quest for a Bulletproof UFC Model

January 20, 2025

One of the hardest parts of training a sports model on UFC is the data drift. In 2016 the Vegas odds were only 61% accurate, then in 2024 they were something like 70% accurate. Right now we're seeing a small decline in the accuracy of Vegas. This is all just part of the ebb and flow of variance that also factors in general unpredictability of sports, especially UFC.

```

LEAST(GREATEST( (observed_strikes - (oh.n_eff_strikes/(oh.n_eff_strikes +
K_mean_strikes) * oh.opp_strikes_mean + (K_mean_strikes/(oh.n_eff_strikes +
K_mean_strikes)) * wc.strikes_mean)) /
GREATEST(oh.n_eff_strikes/(oh.n_eff_strikes + K_mad_strikes) *
oh.opp_strikes_mad + (K_mad_strikes/(oh.n_eff_strikes + K_mad_strikes)) *
wc.strikes_mad, mad_floor), -7), 7) ELSE (observed_strikes - wc.strikes_mean)
/ GREATEST(wc.strikes_mad, mad_floor) END AS strikes_adjperf, -- Similar
pattern for grappling with grappling-specific parameters CASE WHEN
oh.n_eff_grappling >= 1 THEN -- Use grappling-specific K values and opponent
history ELSE -- Fall back to weight-class only END AS grappling_adjperf FROM
fight_stats fs LEFT JOIN opponent_history_per_column oh USING (fight_id,
fighter_id) JOIN weight_class_priors wc ON fs.weightclass = wc.weightclass ) --
Typical K values I use: -- K_mean_strikes = 8, K_mad_strikes = 12 --
K_mean_grappling = 5, K_mad_grappling = 8 -- Higher K_mad provides more
stability for variance estimates

```

Critical implementation insights:

- **Per-column effective n:** Compute *per-column* `n_eff` and set weights to zero when that column has no opponent history, instead of `COALESCE(..., 0)` pretending it does.
- **Global priors as safety net:** Always have a global fallback row in weight-class prior CTEs so sparse classes never collapse toward zero.
- **Per-stat K tuning:** Tune `K_mean` vs `K_mad` per stat family. Generally `K_mad > K_mean` for stability with small samples.
- **Time decay implementation:** Use `EXP(-λ * years_ago)` with $\lambda \approx 0.13$ for moderate decay over ~5 year half-life.

2) Poisson-Gamma Smoothing (Counts)

What was wrong before

- **Under-specified exposure:** I mixed count smoothing and rate logic inconsistently. The posterior mean was formed from (mean, variance) but I didn't consistently convert through exposure time when mapping back to counts. Short fights could look "low activity" for the wrong reason.
- **Ad-hoc variance branch:** When $\text{var} \leq \text{mean}$ I used a hand-rolled blend (e.g., $(\text{prior} \times 3 + \text{observed}) / 4$). It stabilized things but it was a heuristic—not a model.
- **One-size priors by year slice:** Priors came from a fixed date window with mean/variance; no explicit rate per minute, and no principled pseudo-exposure strength.
- **Binary/duration leakage risk:** The old pass didn't clearly wall off duration (control time) or binary signals from count smoothing; at best they were excluded by name heuristics.

What I do now

- **Rate-based Bayesian updating:** I compute weight-class *rates per minute* (μ_w) and update with exposure time t (minutes). The posterior is $\lambda_{\text{post}} = (\mu_w * \tau + X) / (\tau + t)$, and I map back to counts via $X_{\text{smooth}} = t * \lambda_{\text{post}}$.
- **Validated pseudo-minutes:** τ (the prior strength) is per stat and sometimes per weight class (only when cross-validation showed $\geq 0.5\%$ lift). Otherwise I fall back to global τ for consistency.
- **Explicit exposure rules:** Round-1 columns use $\min(\text{time_sec_rd1}, 300) / 60$; everything else uses $\text{time_sec} / 60$. No more implicit exposure guessing.
- **Strict scope:** Count data only (e.g., *_land, *_att, kd, rev). Binary and duration stats are handled elsewhere.

Implementation Details for Your Own Model

Here's the specific SQL pattern I use for Poisson-Gamma smoothing:

```

-- Step 1: Compute weight-class rate priors
WITH wc_priors AS (
    SELECT weightclass,
           SUM(stat_count) / NULLIF(SUM(time_sec / 60.0), 0) AS wc_rate
    FROM fight_stats
    WHERE time_sec > 0
    GROUP BY weightclass
),

-- Step 2: Apply Poisson-Gamma smoothing
smoothed AS (
    SELECT fight_id, fighter_id,
           -- Posterior rate: (prior_rate * pseudo_minutes + observed_count) / (tau + pseudo_minutes)
           (p.wc_rate * tau + fs.stat_count) / (tau + fs.time_sec / 60.0) AS rate,
           -- Convert back to smoothed count
           (fs.time_sec / 60.0) *
           ((p.wc_rate * tau + fs.stat_count) / (tau + fs.time_sec / 60.0)) AS count
    FROM fight_stats fs
    JOIN wc_priors p ON fs.weightclass = p.weightclass
)

-- Key insight: tau values I use:
-- Striking stats: tau = 15-25 minutes
-- Grappling stats: tau = 8-12 minutes
-- Submission attempts: tau = 5-8 minutes

```

Critical implementation notes:

- **Filter out zero exposure:** Always add `WHERE time_sec > 0` when computing priors to avoid division by zero.
- **Round-1 exposure cap:** For first-round stats, cap exposure at 300 seconds (5 minutes) since rounds can't exceed this.
- **Pseudo-minute tuning:** Start with $\tau=15$ for most stats, then use cross-validation to optimize. Higher τ = more shrinkage toward prior.
- **Weight-class fallbacks:** Always have a global prior for weight classes with insufficient data.

Why this fixes the old issues

- **Short/long fights handled correctly:** A two-minute brawl vs. a fifteen-minute chess match are now comparable because smoothing happens on rates and returns properly scaled counts.
- **Less hand-waving, more model:** Pseudo-minutes encode prior confidence without ad-hoc branches; per-class overrides exist only where they earned their keep.

3) Beta-Binomial Smoothing (Binary)

What was wrong before

- **There wasn't any.** KO/Win/Decision/Sub were effectively raw indicators or simple ratios. That's noisy, mis-calibrated, and conflates "skill" with "opportunity."
- **Attempts were undefined:** I had no principled "denominator" for success probability. For example, calling every strike a KO attempt is just wrong.

What I do now

- **Proper Beta-Binomial:** For each binary family I define attempts and successes:
 - `ko/win/decision` : each fight is one opportunity.
 - `sub_land` : opportunities = `sub_att` .
 - `ctrl` (duration): modeled as a time-share (Bernoulli per second); smoothed rate times exposure seconds.
- **Weight-class priors, validated strength:** I use WC success rates as p priors and add pseudo-counts τ tuned per stat, optionally per WC where cross-validation justifies it (e.g., Featherweight subs, LHW/HW control).
- **Zero-attempt guard:** If attempts are zero, I return the WC/global prior rate rather than fabricating a fraction.

Implementation Details for Beta-Binomial Smoothing

- **Control time as time-share:** Model control duration as Bernoulli per second, then multiply smoothed probability by total seconds.
- **Minimum attempts:** Use `GREATEST(attempts, 1)` to avoid division by zero in edge cases.
- **Cross-validation tuning:** Start with τ values above, then optimize per weight class only if CV shows >0.5% improvement.

Why this fixes the old issues

- **Calibration:** Probabilities stop over- or under-shooting because we borrow signal from the division realistically.
- **Correct denominators:** A sub success isn't "one per fight;" it's "out of sub attempts." A KO isn't "per strike;" it's "per fight." This matters.
- **Principled uncertainty:** Small sample sizes get more shrinkage, large samples trust their own data more.

4) Pipeline & Naming Hygiene

- **Order matters:** Binary smoothing runs *before* count smoothing (so sub attempts are raw counts when they need to be). After smoothing, I temporarily keep `*_raw`, compute derived stats (totals, accuracy, defense, ratios, per), then I drop `*_raw`. AdjPerf waits until opponent and weight-class aggregates are ready.
- **Consistent feature families:** Head/Body/Leg and Distance/Clinch/Ground share a naming pattern, so per/ratio/opp calculations can be reliably applied and diffed.

Why this fixes the old issues

- **No accidental toggling:** Derived features don't unknowingly mix raw and smoothed values.
- **Less glue code, fewer footguns:** Calculators can target families by suffix/prefix without bespoke filters for every one-off.

Complete Implementation Roadmap

If you're building your own MMA prediction model, here's the exact order of operations that prevents data leakage and ensures all derived features use properly smoothed inputs:

Step-by-Step Pipeline

1. **Base feature extraction:** Raw fight stats → `fight_stats_derived` table
2. **Beta-Binomial smoothing:** Binary outcomes (KO, win, decision, sub success, control time-share)
3. **Poisson-Gamma smoothing:** Count statistics (strikes, takedowns, etc.)
4. **Temporary raw preservation:** Keep `*_raw` columns during derivation phase
5. **Derived feature computation:** Totals, accuracy, defense rates, ratios - all using smoothed values
6. **Raw column cleanup:** Drop `*_raw` columns after derived features are complete
7. **Per-minute and ratio features:** Rates, pressure metrics, position-specific stats
8. **Feature family tables:** Separate tables for striking, grappling, position stats
9. **Opponent aggregation:** Build opponent history tables with time decay
10. **Weight-class priors:** Compute means, MADs, and minimum MAD floors
11. **Adjusted Performance:** Apply opponent-aware, reliability-weighted standardization

Key SQL Patterns You Can Reuse

```
-- 1. Time decay weight calculation
SELECT EXP(-0.13 * EXTRACT(years FROM current_date - fight_date)) AS dec

-- 2. Effective sample size (Kish formula)
```



```

SELECT POWER(SUM(w), 2) / NULLIF(SUM(POWER(w, 2)), 0) AS n_effective

-- 3. Shrinkage toward prior
SELECT (n/(n + K) * observed + K/(n + K) * prior) AS shrunk_estimate

-- 4. Robust MAD calculation
WITH medians AS (
    SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY stat_value) AS me
    FROM stats_table
),
deviations AS (
    SELECT ABS(stat_value - m.median_val) AS abs_dev
    FROM stats_table s CROSS JOIN medians m
)
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY abs_dev) AS mad_value
FROM deviations

-- 5. Exposure-corrected rate calculation
SELECT stat_count / GREATEST(time_minutes, 0.1) AS rate_per_minute

```

Hyperparameters That Matter Most

- **Time decay λ :** 0.13 gives ~5 year half-life for fight relevance
- **Poisson-Gamma τ (pseudo-minutes):** 15-25 for striking, 8-12 for grappling
- **Beta-Binomial τ (pseudo-attempts):** 8-12 for outcomes, 3-5 for submission success
- **Shrinkage K values:** $K_{\text{mean}} = 5-8$, $K_{\text{mad}} = 8-12$ (higher for stability)
- **Adjusted performance clipping:** $[-7, +7]$ to prevent extreme outliers

Common Pitfalls to Avoid

- **Data leakage:** Always filter opponent history to `event_date < current_fight_date`
- **Zero exposure:** Add `WHERE time_sec > 0` filters when computing rate priors
- **Missing weight classes:** Include global fallback rows in all prior computations

- **Mixed raw/smoothed:** Ensure derived features use smoothed inputs consistently
- **Inadequate sample sizes:** Set minimum thresholds for opponent history reliability

Bottom line: I replaced fragile, one-row expectations and ad-hoc blends with context-aware, exposure-correct, empirically-shrunk estimates. This systematic approach gives you fewer "spikes," better calibration, and more signal per feature. The key is getting the order right and being consistent about shrinkage parameters across your entire pipeline.

Data Drift, Generalization, and the Quest for a Bulletproof UFC Model

January 20, 2025

One of the hardest parts of training a sports model on UFC is the data drift. In 2016 the Vegas odds were only 61% accurate, then in 2024 they were something like 70% accurate. Right now we're seeing a small decline in the accuracy of Vegas. This is all just part of the ebb and flow of variance that also factors in general unpredictability of sports, especially UFC.

For the latest version of the model I spent a ton of time trying to really generalize the model so that I'm not overfitting to specific circumstances at any small point in time. These are the parameters I ended up settling on after hundreds of experiments:

```
train_size = 0.75
val_size = 0.15
test_size = 0.1
n_splits = 4
```

```
num_stack_levels = 2
use_recency_weights = True
use_bag_holdout = True # Must be true if we're using tuning_data (val sp
num_bag_sets = 2
decay_rate = 0.13
shuffle = True
start_date = '2014-04-01'
calibrate = True
```

Breaking Down the AutoGluon Parameters

Let me explain each of these settings in simple terms:

- **train_size/val_size/test_size:** We use 75% for training, 15% for calibration validation, and 10% for final testing—larger validation than test because calibration requires substantial data to work properly.
- **n_splits:** Cross-fold validation uses 4 splits on the training data to ensure robust model selection.
- **num_stack_levels:** AutoGluon stacks models in 2 layers, where second-layer models learn from first-layer predictions.
- **use_recency_weights:** More recent fights are weighted heavier in training to capture current fighting trends.
- **decay_rate:** At 0.13, the earliest fight from 2014 weighs about 0.5 while the latest fight weighs about 1.5—making recent fights 3x more important.
- **use_bag_holdout/num_bag_sets:** Creates multiple model bags with holdout validation for better ensemble diversity.
- **shuffle:** Randomizes training order to prevent temporal bias during model training.
- **calibrate:** Applies post-hoc probability calibration to improve confidence score reliability.

For a long time I was doing a Cross-Fold Validation on 90% of the data, then the last 10% (about 1 year of the most recent fights) was left unseen as an impartial test of how well the model is generalizing to unseen future fights.

4-fold validation (k=4)



Cross-validation ensures robust model evaluation by training and testing on different data splits, helping prevent overfitting and providing more reliable performance estimates.

The Evolution of Validation Strategy

The reason for this was I wanted to maximize the most recent fight data to better predict near future results. As the years have gone by and I've gained a better overhead view of general variance in this sport, I'm slightly leaning towards really thinking through the architecture with generalization in mind will perform better over the long term rather than trying to maximize training data to the most recent fights. But like everything else in this journey, I'll experiment with this strategy and possibly revert back depending on how things go in the testing and real world as time goes on.

Once I started messing with calibration again I was required to go back to a train/val/test method where we do CFV on the train set then calibrate the predictions on val and get an unbiased evaluation of the model from unseen data on test.

So 2014-2023 is training, 2024 is calibration, and 2025 is test (more or less). Over hundreds of tweaks to the parameters above, this final set of parameters is showing the most consistency. I'm very sensitive to not just use the model that had the best results on the validation and test results because that can lead to overfitting where we just tuned parameters to maximize performance on the

training and validation sets. Instead, we make sure that there isn't too much difference between validation and test set performance and that the training set performance isn't wildly higher than val/test. Results:

```
Training accuracy: 0.7511
Training log loss: -0.5082
Test accuracy: 0.7008
Test log loss: -0.5949
Val accuracy: 0.7072
Val log loss: -0.5918
```

Very nice sub -0.6 logloss and >70% accuracy on both the validation set and the unseen test set. Just to make sure we're not overfitting the model I started doing experiments where I use the same parameters and train the model based on cutoff dates, so how would these parameters perform if it didn't have access to the last 6 months of data or the last 12?

The P-Hacking Trap

Same parameters using data with a cutoff of 6 months ago:

```
Training accuracy: 0.7874
Training log loss: -0.4849
Test accuracy: 0.7149
Test log loss: -0.5785
Val accuracy: 0.6542
Val log loss: -0.6044
```

Reasonable. So I feel pretty good about the generalizability of the parameters I'm using right now and the training method implemented. I can p-hack the shit out of this though. For those unfamiliar, p-hacking is a term used in statistics where you "massage" the data or process so it looks like you're measuring some metric in a reasonable way but in fact you're just tweaking small portion of the stats so that the final benchmark metric like accuracy or logloss is maximized to make yourself look smart but isn't accurate to real-world measurement. For instance, what happens if I make a tiny change like increasing the initial data cutoff by 4 months from 2014-01-01 to 2014-04-01?

Training accuracy: 0.8460

Training log loss: -0.4122

Val accuracy: 0.7056

Val log loss: -0.5895

Test accuracy: 0.7121

Test log loss: -0.5864

Wowie weewah I improved the validation and test set logloss AND accuracy! I'm a motherfucking genius. This kind of improvement is unlikely to generalize to real world increases of accuracy and logloss. Look at the fact that the training accuracy increased by 6% leading to a larger gap between the training accuracy/logloss and the val/test logloss/accuracy. This is a negative improvement that likely means the model is a little bit more overfitted to the historical data.

You could argue this isn't that relevant since the unseen fight data accuracy and logloss improved and you wouldn't necessarily be wrong, but here in lies the difficulty of machine learning: you're always backtesting and there is no way to look into the future and tell how the model will actually perform in the real world, all you can do is decipher clues and the clue that stands out to me is a giant 6% leap in training data accuracy lead to an almost 15% gap between training accuracy and val/test accuracy. This is not good because we will never see 85% accuracy in unseen data, ever. We are straying further from Jesus with this minor change. We could've just eliminated a chunk of high variance fights where a bunch of unpredicted big underdogs won but by eliminating these high variance fights, we might've harmed the model's ability to recognize patterns in high variance fights that are still likely to occur in the future and so harmed the generalized ability of the model to predict future fights.

Why Not Include Odds as Features Anymore?

Because the odds accuracy vary wildly compared to more predictable sports like MLB, Soccer, or NFL. Again, the odds were 61% accurate in 2016 yet 70% accurate in 2024. Including the odds essentially makes the model subject to the whims of vegas rather than concretely generalizable over the long term. Second, since we have such highly engineered features, including the odds barely increases the accuracy of the model although it does improve the logloss quite well. All my odd-included models are -ROI on all down-the-line AI

predictions because it so heavily favors betting the vegas odds favorite. It is useful still as a secondary measurement of risk-adjusted returns on favorites, but ultimately it's not that useful in generalized risk-adjusted returns on predictions.

Betting Strategy Performance Analysis

The real test of any model isn't just accuracy—it's profitability. Here's how different betting strategies performed over the latest backtest period, starting with \$1,000 and betting \$10 per pick:

Key Performance Metrics Explained

- **ROI (%)**: Return on investment, measuring profit as a percentage of total amount wagered.
- **Sharpe (ann.)**: Risk-adjusted return that accounts for volatility—higher values indicate better risk-adjusted performance.
- **Sortino (ann.)**: Similar to Sharpe but only penalizes downside volatility, focusing on harmful risk.
- **CAGR (%)**: Compound Annual Growth Rate, showing how fast your bankroll would grow annually.
- **Max DD (%)**: Maximum drawdown, the largest peak-to-trough decline in bankroll value.
- **Calmar**: CAGR divided by maximum drawdown, measuring return per unit of downside risk.
- **PF**: Profit Factor, the ratio of gross profits to gross losses.
- **ROI-Sharpe**: A custom metric combining ROI and Sharpe ratio for overall strategy evaluation.

Backtest Summary (2024-08-03 to Present)

Test Period: Starting from August 3, 2024 with \$1,000 initial bankroll and \$10 bet size

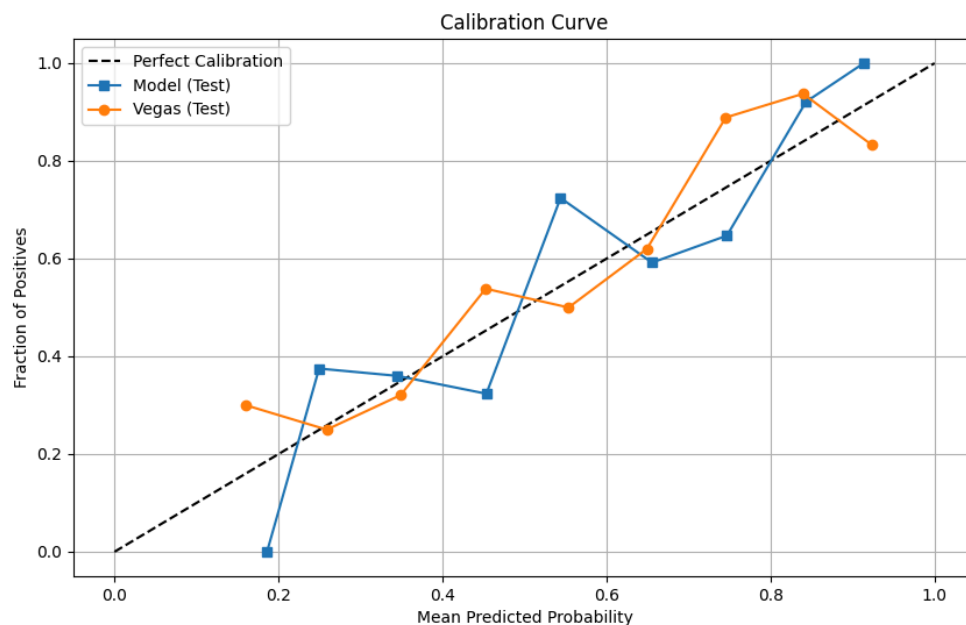
 Best Overall: ai_all_picks_sevenday**ROI:** 10.87% | **Sharpe:** 2.11 | **Final Bankroll:** \$1,287.02 **Closing Odds Strategy****ROI:** 9.51% | **Sharpe:** 1.83 | **Final Bankroll:** \$1,250.98 **Edge Threshold Strategy****ROI:** 3.68% | **Sharpe:** 0.47 | **Final Bankroll:** \$1,086.01

Complete Performance Data: For detailed metrics including CAGR, maximum drawdown, Sortino ratios, and more, download the full backtest results:

 [Download Complete Backtest Data \(TXT\)](#) [Download Complete Backtest Data \(CSV\)](#)

Model Calibration Analysis

Beyond profitability metrics, it's crucial to understand how well our AI model's confidence levels align with actual outcomes. A calibration curve shows whether the model's predicted probabilities match real-world frequencies—if the AI says a fighter has a 70% chance of winning, do they actually win about 70% of the time?



Calibration curve showing the relationship between predicted probabilities and actual outcomes. A perfectly calibrated model would follow the diagonal line—deviations indicate overconfidence (above line) or underconfidence (below line) in predictions.

The calibration curve reveals how reliable our model's confidence estimates are across different probability ranges. This is essential for betting strategies because misaligned confidence can lead to poor risk assessment—betting too heavily on picks the model is overconfident about, or missing value when the model is underconfident.

Key Insights from the Strategy Analysis

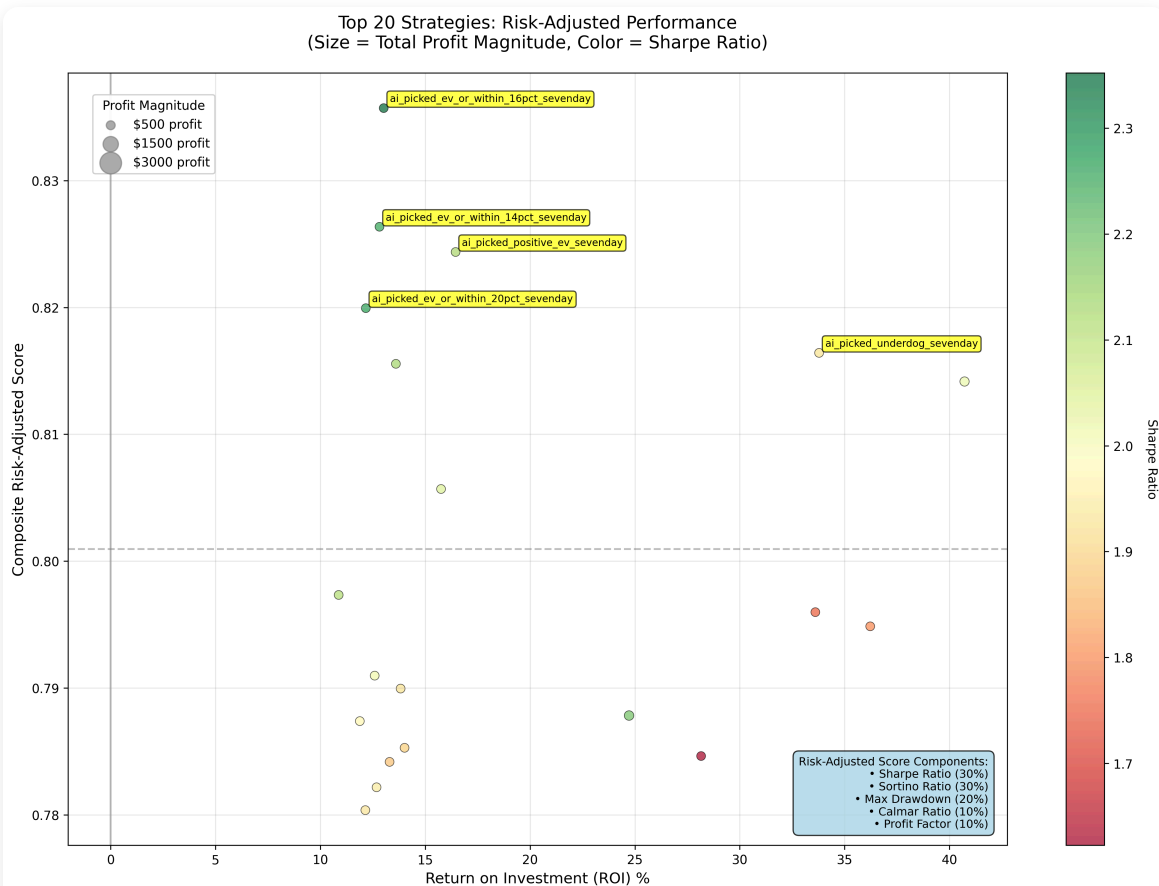
Lines generally move against us: The fact that the seven-day strategy consistently outperforms closing odds betting is a GREAT sign that the model is a sharp picker. When books adjust lines toward our picks by fight night, it validates our edge.

Risk-adjusted returns matter most: For gamblers, the most important metric isn't ROI, accuracy, logloss, or even expected value alone—it's risk-adjusted returns. You want sustainable profit without excessive volatility that can bankrupt you during inevitable losing streaks.

Based on the comprehensive analysis, the optimal strategy appears to be betting all AI picks within a 16% difference between the AI win probability and Vegas implied probability. For example, if the AI picks fighter1 at 70% and Vegas implied odds suggest 84%, we still bet on fighter1. Beyond this 16% threshold, we should consider placing half-unit bets AGAINST the fighter the AI picked.

Risk-Adjusted Strategy Performance

First, thanks to the community for helping me learn here (Shoutout to Jordan C. and the rest of you). To better understand the relationship between risk and reward across different betting strategies, we've analyzed the top 20 performing strategies based on their risk-adjusted returns. The scatter plot below shows each strategy's return on investment (ROI) versus its Sharpe ratio, which measures risk-adjusted performance.



Scatter plot showing ROI vs Sharpe ratio for the top 20 betting strategies. Strategies in the upper-right quadrant offer the best combination of high returns and good risk-adjusted performance.

The ideal strategies appear in the upper-right quadrant, offering both high ROI and strong risk-adjusted returns (high Sharpe ratio). This visualization helps identify strategies that not only generate profit but do so with manageable risk and volatility. Most of the "bet against AI pick based on (AI win% - Vegas odds implied probability) strategies have a cutoff of about 16-20%. Meaning the right time to bet against the AI profitably is when AI win% is at least 16-20% lower than the Vegas implied odds difference and even then it's probably not worth more than 1/2 a unit. The model appears to be very good at predicting underdogs at about a 50% rate, but it's general ability to accurately predict win% is still not as good as Vegas meaning betting solely on +EV isn't necessarily the best strategy.

Conclusion and Call for Feedback

Two important points:

1) **Please be critical of this and send me unfiltered opinions and help.** I have nowhere near the level of experience in picking betting strategies as I do in training models. The machine learning side I've got down, but translating statistical edges into optimal betting strategies is where I need the most improvement.

2) The data suggests our best approach is the nuanced edge-threshold strategy rather than blindly following all AI picks or only +EV selections. This makes intuitive sense—we trust the model most when it aligns reasonably well with market expectations, but we hedge when there's significant disagreement.

As always, this is an ongoing experiment. The beauty and curse of sports prediction is that the landscape constantly shifts. What works today might not work tomorrow, which is why building robust, generalizable systems matters more than chasing short-term optimization.

Get in Touch: The best place to reach me is on [Patreon](#) where the community shares free UFC predictions and discuss model development. You don't need to subscribe to participate in chats and direct messages—it's the most active community for discussing these predictions and sharing feedback. I'd love to hear your thoughts on the strategies and any suggestions for improvement!

Calibrating UFC Fight Predictions

July 15, 2025

Introduction

My UFC prediction model has maintained profitable performance for years despite having poorly calibrated probability estimates. While the model excels at binary classification (picking winners), its confidence scores don't align well with real-world win frequencies—a classic case of good accuracy, bad calibration.

This never bothered me much since the model was profitable, but I decided to experiment with post-hoc calibration methods to see if I could improve probability estimates without hurting classification performance. This post documents those experiments: the methods I tested, why most failed with limited data, and the modest improvements I eventually achieved with Platt scaling.

What is Model Calibration?

Model calibration refers to how well a model's predicted probabilities align with actual observed frequencies. A perfectly calibrated model should be correct 70% of the time when it predicts a 70% probability, 80% of the time when it predicts 80%, and so on.

Consider this example: if your model predicts 100 fights at 60% confidence, and the favored fighter wins 60 of those fights, your model is well-calibrated at that confidence level. However, if the favored fighter wins 75 times, your model is under-confident and poorly calibrated.

```
from sklearn.calibration import calibration_curve
import matplotlib.pyplot as plt

# Example: plotting calibration curve
fraction_pos, mean_pred = calibration_curve(y_true, y_prob, n_bins=10)
```

```
plt.plot([0, 1], [0, 1], 'k--', label='Perfect calibration')
plt.plot(mean_pred, fraction_pos, 's-', label='Model')
plt.xlabel('Mean Predicted Probability')
plt.ylabel('Fraction of Positives')
```

The Calibration vs. Accuracy Paradox

One of the most counterintuitive aspects of calibration is that improving probability estimates can sometimes hurt classification accuracy. This happens because calibration optimizes for probability-based metrics like log-loss and Brier score, while accuracy only cares about the binary decision boundary at 50%.

Why This Happens

Consider a model that consistently predicts 65% when the true probability is 60%. This model might achieve higher accuracy by being overconfident (correctly picking the winner more often due to increased confidence), but it's poorly calibrated. When calibrated, the model's predictions become more conservative, potentially crossing the 50% decision boundary in some cases and reducing accuracy.

In my UFC model, this manifested as:

Original Model Performance:

Accuracy:	0.7098
Log Loss:	0.6032
Brier Score:	0.2075

Calibrated Model Performance:

Accuracy:	0.7098 (unchanged)
Log Loss:	0.5979 (improved by 0.0053)
Brier Score:	0.2056 (improved by 0.0019)

The accuracy remained stable while probability-based metrics improved significantly but that isn't true for all calibration methods and isn't even necessarily true 100% of the time for the method that actually worked.

The Small Sample Size Challenge

My dataset contains approximately 2,400 UFC fights over 10 years after extensive filtering:

```
def filter_fights(df, threshold, date='2015-01-01', include_split_dec=False):
    """
    Filter fights based on:
    - Binary results (y_true in [0, 1])
    - Both fighters must have had at least num_fights previous fights
    - Removing unwanted fight methods (DQ, split decisions, etc.)
    - Fights from 2015 onward
    """
    # Remove unwanted methods
    if include_split_dec:
        unwanted_methods = ['dq', 'other', 'overturned']
    else:
        unwanted_methods = ['dq', 'other', 'decision - split', 'decision']

    # Filter to only binary results and recent fights
    df = df[df['y_true'].isin([0, 1])].copy()
    df = df[df['event_date'] >= pd.Timestamp(date)]

    return df
```

This small sample size created significant challenges for calibration, particularly with isotonic regression which requires sufficient data points across the probability spectrum.

Isotonic Regression: The First Attempt

Isotonic regression is a non-parametric calibration method that learns a monotonic mapping from predicted probabilities to calibrated probabilities. It's theoretically superior to Platt scaling as it can capture non-linear calibration relationships.

```
class SimpleIsotonicCalibration:
    def __init__(self, y_min=0.01, y_max=0.99):
        self.y_min = y_min
        self.y_max = y_max
        self.calibrator = None

    def fit(self, y_prob, y_true):
```

```
from sklearn.isotonic import IsotonicRegression

self.calibrator = IsotonicRegression(
    y_min=self.y_min,
    y_max=self.y_max,
    out_of_bounds='clip'
)

self.calibrator.fit(y_prob, y_true)
```

Isotonic Results (Disappointing)

Isotonic Calibration Results:

Should Use Calibration: False

Calibration Set Log Loss Improvement: 0.0553 # Improvement on the calibr

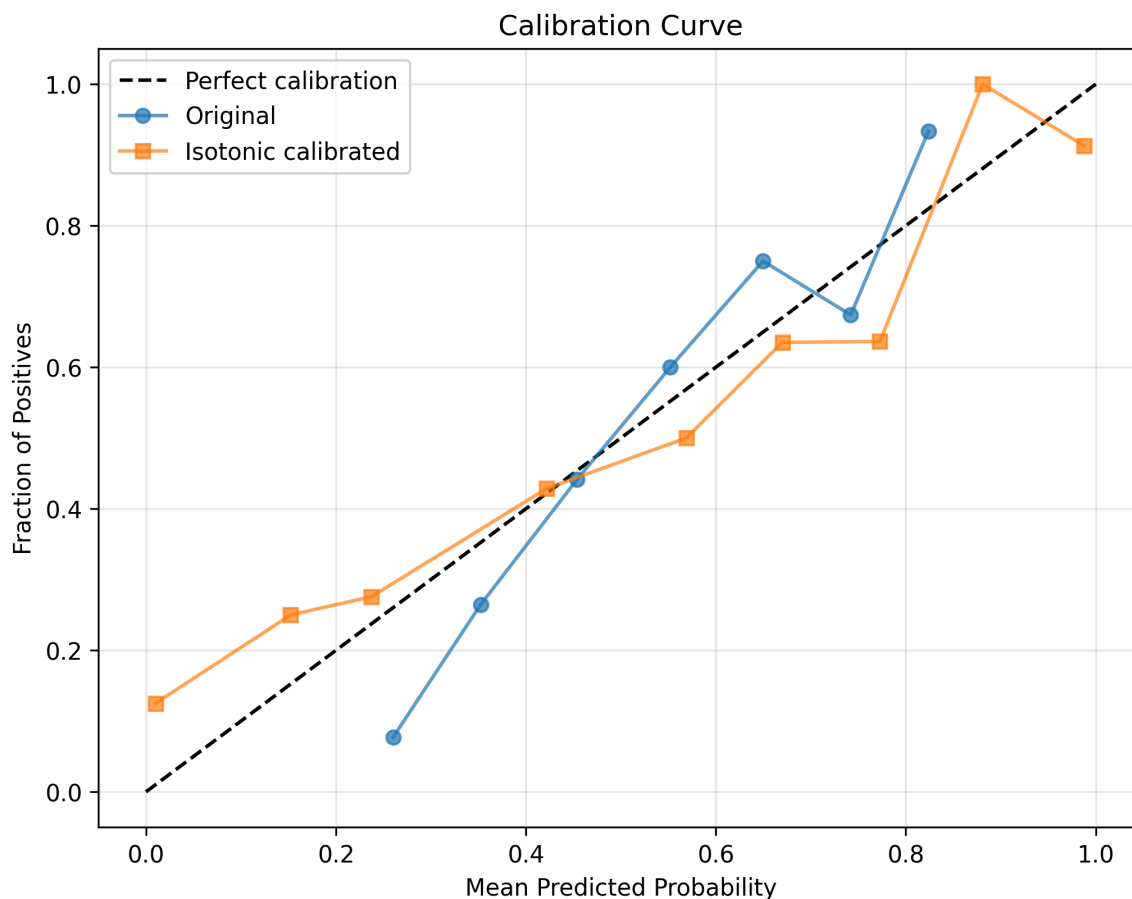
Test Set Log Loss Improvement: 0.0125 # Worse on the unseen test set!

Test Set Brier Score Improvement: 0.0023 # Worse on the unseen test set!

Final Test Log Loss (Original): 0.5937

Final Test Log Loss (Calibrated): 0.6063

The isotonic calibration actually **hurt** performance on the test set despite improving calibration set metrics. This is a classic sign of overfitting due to insufficient data.



Why Isotonic Failed

1. **Small sample size:** With only ~240 test samples (10% of 2,400), isotonic regression had insufficient data to learn a robust monotonic mapping
2. **Sparse probability regions:** Some probability ranges had very few examples, leading to unreliable calibration
3. **Overfitting:** The flexibility of isotonic regression became a liability with limited data

I attempted several improvements:

- **Ensemble approach:** Multiple isotonic regressors trained on different CV folds using all the training data. This was dumb because I was just training the calibration model on fights the main model had already been trained on leading to overfitting on train, and poor results on the holdout test dataset.
- **Expanded calibration set:** Increased from 10% to 20% of data
- **Parameter tuning:** Adjusted `y_min`, `y_max`, and `out_of_bounds` settings

None of these approaches yielded meaningful improvements.

Platt Scaling: A Little Better

Platt scaling uses logistic regression to map uncalibrated probabilities to calibrated ones. While less flexible than isotonic regression, it's much more suitable for small datasets.

```
class SimplePlattCalibration:
    def __init__(self, max_iter=100, random_state=42):
        self.max_iter = max_iter
        self.random_state = random_state
        self.calibrator = None

    def fit(self, y_prob, y_true):
        from sklearn.linear_model import LogisticRegression

        # Reshape probabilities for sklearn (needs 2D input)
        y_prob_reshaped = y_prob.reshape(-1, 1)

        self.calibrator = LogisticRegression(
            max_iter=self.max_iter,
            random_state=self.random_state,
            solver='lbfgs'
        )
        self.calibrator.fit(y_prob_reshaped, y_true)
```

Implementation in Training Pipeline

The calibration was integrated into the training pipeline using scikit-learn's

`CalibratedClassifierCV` :

```
# Three-way split for proper calibration validation
(X_train, y_train), (X_cal, y_cal), (X_test, y_test) = split_data_three_
    X, y, train_size=0.775, val_size=0.125
)

# Wrap AutoGluon predictor for sklearn compatibility
autogluon_wrapper = AutoGluonWrapper(predictor, feature_columns=X_train.

# Create calibrated classifier using holdout method
```

```

calibrated_clf = CalibratedClassifierCV(
    estimator=autogluon_wrapper,
    method='sigmoid', # Platt scaling
    cv="prefit",      # Use prefit since AutoGluon is already trained
    ensemble=False    # Use single calibrator since we have proper spli
)

# Fit calibrator on holdout calibration set
calibrated_clf.fit(X_cal_clean, y_cal)

```

Platt Scaling Results (Success!)

```

Calibration Results (sigmoid):
Should Use Calibration: True
Calibration Set Log Loss Improvement: 0.0098
Test Set Log Loss Improvement: 0.0107
Test Set Brier Score Improvement: 0.0043
Test Set ECE Improvement: 0.0174
Final Test Log Loss (Original): 0.5948
Final Test Log Loss (Calibrated): 0.5841

```

The improvement of 0.0107 in log-loss on the holdout test set of data represents a meaningful, if small, gain in probability accuracy.

Profitability Impact Analysis

While the statistical improvements were modest, I wanted to examine whether calibration affected betting profitability. Here's the cumulative profit comparison between the original and calibrated models on the test set:

Start Date: 2024-06-29 Initial Bankroll: \$1000.00 Bet Amount: \$10.00 -----									
Strategy	Final Bankroll	Total Profit	Total Wagered	Bets	Wins	Win %	ROI (%)	Sharpe Ratio	
ai_all_picks_closing	\$ 1339.33	\$ 339.33	\$ 2560.00	256	181	70.70%	13.26%	0.37	
ai_all_picks_sevenday	\$ 1383.38	\$ 383.38	\$ 2560.00	256	181	70.70%	14.98%	0.42	
ai_picked_ev_or_within_5pct_closing	\$ 1266.44	\$ 266.44	\$ 1470.00	147	91	61.90%	18.13%	0.31	
ai_picked_ev_or_within_5pct_sevenday	\$ 1259.35	\$ 259.35	\$ 1600.00	160	99	61.88%	16.21%	0.29	
ai_picked_favorite_closing	\$ 1123.95	\$ 123.95	\$ 1950.00	195	149	76.41%	6.36%	0.22	
ai_picked_favorite_sevenday	\$ 1125.78	\$ 125.78	\$ 1870.00	187	142	75.94%	6.73%	0.23	
ai_picked_positive_ev_closing	\$ 1172.45	\$ 172.45	\$ 1090.00	109	60	55.05%	15.82%	0.22	
ai_picked_positive_ev_sevenday	\$ 1229.58	\$ 229.58	\$ 1300.00	130	76	58.46%	17.66%	0.26	
ai_picked_underdog_closing	\$ 1215.38	\$ 215.38	\$ 610.00	61	32	52.46%	35.31%	0.34	
ai_picked_underdog_sevenday	\$ 1277.60	\$ 277.60	\$ 670.00	67	39	58.21%	41.43%	0.40	
any_fighter_positive_ev_closing	\$ 715.05	\$ -284.95	\$ 2250.00	225	80	35.56%	-12.66%	-0.20	
any_fighter_positive_ev_sevenday	\$ 706.08	\$ -293.92	\$ 2290.00	229	90	39.30%	-12.83%	-0.22	
bet_against_ai_if_opponent_5pct_edge_closing	\$ 956.90	\$ -43.10	\$ 2560.00	256	126	49.22%	-1.68%	-0.03	
bet_against_ai_if_opponent_5pct_edge_sevenday	\$ 953.28	\$ -46.72	\$ 2560.00	256	133	51.95%	-1.82%	-0.04	

Start Date: 2024-06-29
Initial Bankroll: \$1000.00
Bet Amount: \$10.00

Strategy	Final Bankroll	Total Profit	Total Wagered	Bets	Wins	Win %	ROI (%)	Sharpe Ratio
ai_all_picks_closing	\$ 1348.13	\$ 348.13	\$ 2560.00	256	181	70.70%	13.60%	0.38
ai_all_picks_sevenday	\$ 1391.75	\$ 391.75	\$ 2560.00	256	181	70.70%	15.30%	0.43
ai_picked_ev_or_within_5pct_closing	\$ 1286.77	\$ 286.77	\$ 1920.00	192	127	66.15%	14.94%	0.32
ai_picked_ev_or_within_5pct_sevenday	\$ 1308.04	\$ 308.04	\$ 1980.00	198	130	65.66%	15.56%	0.35
ai_picked_favorite_closing	\$ 1127.85	\$ 127.85	\$ 1930.00	193	148	76.68%	6.62%	0.24
ai_picked_favorite_sevenday	\$ 1131.15	\$ 131.15	\$ 1850.00	185	141	76.22%	7.09%	0.25
ai_picked_positive_ev_closing	\$ 1248.10	\$ 248.10	\$ 1570.00	157	98	62.42%	15.80%	0.29
ai_picked_positive_ev_sevenday	\$ 1282.75	\$ 282.75	\$ 1710.00	171	109	63.74%	16.54%	0.33
ai_picked_underdog_closing	\$ 1220.28	\$ 220.28	\$ 630.00	63	33	52.38%	34.97%	0.33
ai_picked_underdog_sevenday	\$ 1280.60	\$ 280.60	\$ 690.00	69	40	57.97%	40.67%	0.40
any_fighter_positive_ev_closing	\$ 992.40	\$ -7.60	\$ 2230.00	223	108	48.43%	-0.34%	-0.01
any_fighter_positive_ev_sevenday	\$ 952.95	\$ -47.05	\$ 2300.00	230	116	50.43%	-2.05%	-0.04

The results show modest but consistent improvements across most betting strategies. The core strategy (ai_all_picks_closing) improved from 13.26% to 13.60% ROI, while the seven-day advance strategy increased from 14.98% to 15.30% ROI. Notably, win percentages remained identical at 70.70% which is interesting because some picks do change based on the calibration process.

Interestingly, underdog-focused strategies saw slight decreases in ROI (35.31% to 34.97% for closing odds) and slightly improved favorite win percentages (70.70% to 70.80%), suggesting the calibration process may have made the model slightly more conservative on high-value underdog picks. Meanwhile, the poorly performing +EV on ANY fighter (regardless of AI pick) strategies remained unprofitable in both versions but dramatically improved in the calibrated version showing the calibration process working in action. The AI-picked +EV strategy was basically the same though which tells me I need more profit testing on where the +EV threshold is, like +5% win chance? +10% win chance? When do we bet against the AI pick? Like last event Kevin Holland was picked by AI at -180 or something yet Vegas had him at -500. What's the threshold to pick against the AI? IDK yet but I'll go test it out. This is the most pressing question when we're talking about betting strategy.

However, it's crucial to note that ROI is a moving target—betting markets evolve, line movement varies, and small sample sizes can significantly impact results. These profit tests represent performance on a specific test set and shouldn't be viewed as guaranteed future returns. The real value of calibration lies in more reliable probability estimates for bet sizing and strategy decisions rather than raw profit maximization.

Model Performance Analysis

Here's how the calibrated model performed against Vegas odds across all unfiltered fights in the past 1.5 years (meaning we include split decisions, and

DQ's, and whatnot which aren't included in the model training):

```
{
  "vegas_odds_performance": {
    "accuracy": 0.700,
    "log_loss": 0.563,
    "brier_score": 0.194
  },
  "mma_ai_performance": {
    "accuracy": 0.710,
    "log_loss": 0.603,
    "brier_score": 0.208
  },
  "mma_ai_performance_calibrated": {
    "accuracy": 0.710,
    "log_loss": 0.598, # Improved
    "brier_score": 0.206 # Improved
  }
}
```

The calibrated model maintains identical accuracy while providing more reliable probability estimates.

The Calibration Curve Analysis

The most telling evidence comes from the calibration curve itself. My uncalibrated model exhibited classic miscalibration patterns:

- **Underconfident in 50-60% range:** When the model predicted 55%, fighters actually won ~60% of the time
- **Overconfident in 40-50% range:** When the model predicted 45%, fighters won closer to 40% of the time

```
def plot_calibration_curve(self, n_bins=10, include_all_fights: bool = True):
    # Get calibrated predictions if calibrator is available
    y_prob_calibrated = None
    if self.calibrator is not None:
        y_prob_calibrated = self.calibrator.predict_proba(test_data_clean)

    # Calculate calibration curves
```

```

prob_true_model, prob_pred_model = calibration_curve(self.y_test, y_
if y_prob_calibrated is not None:
    prob_true_calibrated, prob_pred_calibrated = calibration_curve(
        self.y_test, y_prob_calibrated, n_bins=n_bins
    )

# Plot results
plt.plot([0, 1], [0, 1], 'k--', label='Perfect Calibration')
plt.plot(prob_pred_model, prob_true_model, 's-', label='Model (Original)')
if y_prob_calibrated is not None:
    plt.plot(prob_pred_calibrated, prob_true_calibrated, '^-',
             label='Model Calibrated', alpha=0.8)

```

The Odds Inclusion Paradox

An interesting discovery was the relationship between including betting odds and calibration. In previous experiments 1-2 years ago, calibration consistently hurt performance. The key difference was I was including Vegas odds as features.

With Odds (Historical)

- **Pros:** Exceptional calibration, higher accuracy (~73-74%)
- **Cons:** Model HEAVILY favored odds over other features, struggled with underdog picks, -6% lower ROI against all model picks

Without Odds (Current)

- **Pros:** Better underdog detection, higher betting ROI, room for calibration improvement
- **Cons:** Lower accuracy (~71%), requires manual calibration

This represents a fascinating tradeoff: **accuracy vs. profitability**. The model without odds hits significantly more profitable underdog picks, even though its raw accuracy is lower.

Why Calibration Matters for Bettors

While calibration may not be necessary for a profitable model (mine was already profitable before calibration), it provides crucial benefits for bet sizing and strategy:

Kelly Criterion Application

With well-calibrated probabilities, bettors can use the Kelly Criterion more effectively:

```
def kelly_bet_size(prob, odds, bankroll):  
    """Calculate optimal bet size using Kelly Criterion"""  
    decimal_odds = american_to_decimal(odds)  
    edge = prob * decimal_odds - 1  
    if edge <= 0:  
        return 0  
    kelly_fraction = edge / (decimal_odds - 1)  
    return min(kelly_fraction * bankroll, bankroll * 0.1) # Cap at 10%
```

I experimented with fractional kelly in the past. It's promising long term, especially since the model's logloss is consistently less than .059, but the wild swings are too painful. If we got the model to be lower logloss than Vegas by including the odds in the feature set I think this is promising but will experiment more later with that.

Confidence-Based Strategies

Calibrated probabilities enable more sophisticated betting strategies:

```
def edge_based_betting_strategy(predictions, min_edge=0.05):  
    """Select bets based on edge over Vegas odds rather than absolute co  
    betting_opportunities = []  
  
    for fight in predictions:  
        # Get model probability and Vegas implied probability  
        model_prob = fight['model_confidence']  
        vegas_decimal_odds = american_to_decimal(fight['vegas_odds'])  
        vegas_implied_prob = 1 / vegas_decimal_odds  
  
        # Calculate edge (model probability - market probability)  
        edge = model_prob - vegas_implied_prob
```

```

# Only bet if we have a significant edge
if edge >= min_edge:
    kelly_fraction = edge / (vegas_decimal_odds - 1) # Kelly cr
    betting_opportunities.append({
        'pick': fight['fighter_name'],
        'edge': edge,
    })

return betting_opportunities

```

I'll probably implement something like this later on after I do more profitability backtesting with flat units.

Technical Implementation Details

The final calibration system integrates seamlessly with the existing prediction pipeline:

```

def _get_model_predictions(self, test_data, use_calibrated=None):
    """Get model predictions, optionally using calibrator."""
    if use_calibrated is None:
        use_calibrated = self.calibrator is not None

    # Get original predictions
    y_pred = self.predictor.predict(test_data)
    y_prob = self.predictor.predict_proba(test_data)

    # Apply calibration if available
    if use_calibrated and self.calibrator is not None:
        test_data_clean = test_data.drop(columns=['sample_weight'], erro
        y_prob = self.calibrator.predict_proba(test_data_clean)[: , 1]
        y_pred = (y_prob > 0.5).astype(int)

    return y_pred, y_prob

```

Lessons Learned

1. **Dataset size matters:** Isotonic regression requires substantial data; Platt scaling works better with limited samples
2. **Validation strategy is crucial:** Proper train/calibration/test splits prevent overfitting

3. **Calibration $\neq$ Accuracy:** Better probabilities don't always mean better classifications
4. **Feature engineering impacts calibration:** Including odds improves calibration but hurts profitability
5. **Domain expertise guides tradeoffs:** Understanding the betting market informed the decision to exclude odds

Conclusion

While the final improvement in logloss was modest (0.0107), it represents a meaningful step toward more reliable probability estimates.

The key insight is that calibration serves different purposes depending on your goals. For pure classification accuracy, it is unnecessary. For models that are already well calibrated (like when you include the odds, or use algos that are better for calibration like Random Forest) calibrating the predictions using Platt or Isotonic harms the output.

The surprising relationship between feature inclusion (odds) and calibration highlights the complex tradeoffs in machine learning systems. Sometimes the most accurate model isn't the most profitable one, and the most calibrated model isn't the most accurate one.

For practitioners working with limited datasets, Platt scaling offers a robust path to improved calibration. The simplicity of logistic regression makes it both interpretable and reliable, even when more sophisticated methods fail.

Machine Learning for Sports Prediction: Should You Balance the Winrate of Competitor 1 vs Competitor 2?

June 19, 2025

Not at this time. I've spent so many hours investigating this in my free time. ROI is our north star, with logloss and accuracy being secondary metrics. This isn't a perfect test because the betting markets shift as time moves. For example, 2016 saw Vegas favorites win only 61% of the time. 2024 saw favorites win 70% of the time and UFC red corner is usually the betting odds favorite. So this isn't the end-all be-all conclusion, but it is absolutely the point in time conclusion.

The long and short of the massive amount of trial and error is this backtested ROI based on the last year of fights the model has never seen:

ROI Comparison: Balanced vs Unbalanced Training

ROI of balanced 50/50 fighter1 win/lose:

Initial Bankroll: \$1000.00 Bet Amount: \$10.00									
Strategy	Final Bankroll	Total Profit	Total Wagered	Bets	Wins	Win %	ROI (%)	Sharpe Ratio	
ai_all_picks_closing	\$ 1047.03	\$ 47.03	\$ 2560.00	256	171	66.80%	1.84%	0.05	
ai_all_picks_sevenday	\$ 1105.84	\$ 105.84	\$ 2560.00	256	171	66.80%	4.13%	0.12	
ai_picked_ev_or_within_5pct_closing	\$ 1021.73	\$ 21.73	\$ 1330.00	133	75	56.39%	1.63%	0.03	
ai_picked_ev_or_within_5pct_sevenday	\$ 1099.01	\$ 99.01	\$ 1380.00	138	82	59.42%	7.17%	0.13	
ai_picked_favorite_closing	\$ 1016.27	\$ 16.27	\$ 2090.00	209	151	72.25%	0.78%	0.03	
ai_picked_favorite_sevenday	\$ 1016.38	\$ 16.38	\$ 2010.00	201	144	71.64%	0.81%	0.03	
ai_picked_positive_ev_closing	\$ 994.60	\$ -5.40	\$ 920.00	92	46	50.00%	-0.59%	-0.01	
ai_picked_positive_ev_sevenday	\$ 1066.92	\$ 66.92	\$ 1080.00	108	59	54.63%	6.26%	0.09	
ai_picked_underdog_closing	\$ 1030.76	\$ 30.76	\$ 470.00	47	20	42.55%	6.54%	0.05	
ai_picked_underdog_sevenday	\$ 1109.46	\$ 109.46	\$ 530.00	53	27	50.94%	20.65%	0.17	
any_fighter_positive_ev_closing	\$ 650.20	\$ -349.80	\$ 2170.00	217	73	33.64%	-16.12%	-0.27	
any_fighter_positive_ev_sevenday	\$ 721.32	\$ -278.68	\$ 2270.00	227	88	38.77%	-12.28%	-0.21	
bet_against_ai_if_opponent_5pct_edge_closing	\$ 833.00	\$ -167.00	\$ 2560.00	256	117	45.70%	-6.52%	-0.12	
bet_against_ai_if_opponent_5pct_edge_sevenday	\$ 883.97	\$ -116.03	\$ 2560.00	256	129	50.39%	-4.53%	-0.09	
parlay_2_legs_ai_picked_positive_ev_closing	\$ 1094.50	\$ 94.50	\$ 630.00	63	15	23.81%	15.00%	0.07	
parlay_2_legs_ai_picked_positive_ev_sevenday	\$ 1310.35	\$ 310.35	\$ 820.00	82	26	31.71%	37.85%	0.19	
parlay_2_legs_ai_random_picks_closing	\$ 1141.35	\$ 141.35	\$ 2300.00	230	98	42.61%	6.15%	0.07	
parlay_2_legs_ai_random_picks_sevenday	\$ 1304.43	\$ 304.43	\$ 2300.00	230	106	46.09%	13.24%	0.16	
parlay_2_legs_ai_top_confidence_closing	\$ 1408.03	\$ 408.03	\$ 2300.00	230	116	50.43%	17.74%	0.24	
parlay_2_legs_ai_top_confidence_sevenday	\$ 1539.60	\$ 539.60	\$ 2300.00	230	116	50.43%	23.46%	0.30	
parlay_2_legs_any_fighter_positive_ev_closing	\$ 179.83	\$ -820.17	\$ 1910.00	191	18	9.42%	-42.94%	-0.44	
parlay_2_legs_any_fighter_positive_ev_sevenday	\$ 374.32	\$ -625.68	\$ 1980.00	198	26	13.13%	-31.60%	-0.30	
parlay_3_legs_ai_picked_positive_ev_closing	\$ 1059.17	\$ 59.17	\$ 230.00	23	2	8.70%	25.73%	0.08	
parlay_3_legs_ai_picked_positive_ev_sevenday	\$ 1144.17	\$ 144.17	\$ 360.00	36	5	13.89%	40.05%	0.12	
parlay_3_legs_ai_random_picks_closing	\$ 1209.94	\$ 209.94	\$ 1430.00	143	41	28.67%	14.68%	0.09	
parlay_3_legs_ai_random_picks_sevenday	\$ 1513.47	\$ 513.47	\$ 1430.00	143	45	31.47%	35.91%	0.25	
parlay_3_legs_ai_top_confidence_closing	\$ 1247.13	\$ 247.13	\$ 1430.00	143	45	31.47%	17.28%	0.16	
parlay_3_legs_ai_top_confidence_sevenday	\$ 1305.63	\$ 305.63	\$ 1430.00	143	45	31.47%	21.37%	0.19	

ROI of unbalanced 59/41 fighter1 (UFC assigned red corner) win/lose:

Initial Bankroll: \$1000.00 Bet Amount: \$10.00									
Strategy	Final Bankroll	Total Profit	Total Wagered	Bets	Wins	Win %	ROI (%)	Sharpe Ratio	
ai_all_picks_closing	\$ 1302.04	\$ 302.04	\$ 2500.00	256	180	70.31%	11.80%	0.32	
ai_all_picks_sevenday	\$ 1342.42	\$ 342.42	\$ 2500.00	256	180	70.31%	13.38%	0.35	
ai_picked_ev_or_within_5pct_closing	\$ 1272.75	\$ 272.75	\$ 1470.00	147	93	63.27%	18.55%	0.29	
ai_picked_ev_or_within_5pct_sevenday	\$ 1275.61	\$ 275.61	\$ 1630.00	163	104	63.80%	16.91%	0.29	
ai_picked_favorite_closing	\$ 1110.76	\$ 110.76	\$ 1940.00	194	148	76.29%	5.71%	0.21	
ai_picked_favorite_sevenday	\$ 1107.18	\$ 107.18	\$ 1900.00	190	143	75.26%	5.64%	0.20	
ai_picked_positive_ev_closing	\$ 1236.12	\$ 236.12	\$ 1250.00	125	75	60.00%	18.89%	0.27	
ai_picked_positive_ev_sevenday	\$ 1272.73	\$ 272.73	\$ 1340.00	134	84	62.69%	20.35%	0.31	
ai_picked_underdog_closing	\$ 1191.28	\$ 191.28	\$ 820.00	62	32	51.61%	30.88%	0.28	
ai_picked_underdog_sevenday	\$ 1255.24	\$ 255.24	\$ 640.00	64	37	57.81%	39.88%	0.35	
any_fighter_positive_ev_closing	\$ 852.92	\$ -147.08	\$ 2370.00	237	96	40.51%	-6.21%	-0.10	
any_fighter_positive_ev_sevenday	\$ 877.93	\$ -122.07	\$ 2270.00	227	100	44.05%	-5.38%	-0.09	
bet_against_ai_if_opponent_5pct_edge_closing	\$ 1131.95	\$ 131.95	\$ 2500.00	256	136	53.12%	5.15%	0.09	
bet_against_ai_if_opponent_5pct_edge_sevenday	\$ 1063.97	\$ 63.97	\$ 2500.00	256	139	54.30%	2.50%	0.04	
parlay_2_legs_ai_picked_positive_ev_closing	\$ 1695.29	\$ 695.29	\$ 960.00	96	41	42.71%	72.43%	0.37	
parlay_2_legs_ai_picked_positive_ev_sevenday	\$ 1871.36	\$ 871.36	\$ 1080.00	108	50	46.30%	80.68%	0.48	
parlay_2_legs_ai_random_picks_closing	\$ 1452.74	\$ 452.74	\$ 2300.00	230	108	46.96%	19.68%	0.24	
parlay_2_legs_ai_random_picks_sevenday	\$ 1820.55	\$ 820.55	\$ 2300.00	230	113	49.13%	35.68%	0.36	
parlay_2_legs_ai_top_confidence_closing	\$ 1472.28	\$ 472.28	\$ 2300.00	230	115	50.00%	20.53%	0.25	
parlay_2_legs_ai_top_confidence_sevenday	\$ 1586.91	\$ 586.91	\$ 2300.00	230	115	50.00%	25.52%	0.31	
parlay_2_legs_any_fighter_positive_ev_closing	\$ 790.66	\$ -209.34	\$ 2140.00	214	35	16.36%	-9.78%	-0.08	
parlay_2_legs_any_fighter_positive_ev_sevenday	\$ 911.07	\$ -88.93	\$ 1990.00	199	39	19.60%	-4.47%	-0.04	
parlay_3_legs_ai_picked_positive_ev_closing	\$ 1808.37	\$ 808.37	\$ 500.00	50	16	32.00%	161.67%	0.42	
parlay_3_legs_ai_picked_positive_ev_sevenday	\$ 2088.35	\$ 1088.35	\$ 550.00	55	20	36.36%	197.88%	0.50	
parlay_3_legs_ai_random_picks_closing	\$ 1677.35	\$ 677.35	\$ 1430.00	143	43	30.07%	47.37%	0.29	
parlay_3_legs_ai_random_picks_sevenday	\$ 2090.06	\$ 1090.06	\$ 1430.00	143	51	35.66%	76.23%	0.41	
parlay_3_legs_ai_top_confidence_closing	\$ 1538.08	\$ 538.08	\$ 1430.00	143	50	34.97%	37.63%	0.25	
parlay_3_legs_ai_top_confidence_sevenday	\$ 1608.41	\$ 608.41	\$ 1430.00	143	50	34.97%	42.55%	0.29	

Based on how I do feature engineering and model tuning, balancing the fighters' win rate before training the model is terrible. If I do some various feature and model tuning I can see the balanced model start to perform closer to the unbalanced model but on average it is basically always lower ROI. I have interrogated my code a thousand times with Claude4 Opus and Gemini 2.5 Pro to try to suss out any logical errors and I cannot find any.

Why Does Balanced Training Destroy Performance?

Begin speculation

Distribution shift between training and inference creates systematic probability miscalibration. When you balance the dataset to 50/50, you're teaching the model that $P(\text{fighter1_wins}) = 0.5$ across all feature combinations. But in reality, fighter1 (red corner) wins ~60% of the time because the UFC systematically assigns the red corner to champions in title fights and generally more experienced/favored fighters in regular bouts.

The calibration mismatch:

- **Training:** Model learns $P(\text{fighter1_wins} \mid \text{features})$ where the base rate is artificially 0.5
- **Inference:** Model predicts on data where the true base rate is 0.6

This specifically destroys betting ROI because:

- **Systematic underestimation of favorites:** When a red corner fighter should win with 70% probability, your balanced model might predict 60%, causing you to miss profitable favorite bets
- **Systematic overestimation of underdogs:** When a blue corner fighter should win with 30% probability, your model might predict 40%, leading to negative EV underdog bets
- **Market inefficiency amplification:** The higher ROI without balancing implies that the model is learning this pattern of corner bias more efficiently than the market is.

The model's probability outputs are fundamentally miscalibrated for the real world's corner assignment bias. You're essentially training a model for a balanced fantasy UFC and then applying it to the systematically biased real UFC, where corner assignments carry predictive information about fight outcomes.

Bottom line: The balanced training throws away valuable signal (red corner = usually stronger fighter) and teaches the model incorrect base rates, leading to systematically poor probability estimates that destroy betting performance.

Doesn't Calibrating the Winrate Just Create a Proxy for the Odds?

No, and here's why that concern misses the point:

The model isn't learning "red corner = bet favorite." It's learning complex feature interactions from granular performance statistics that happen to correlate with corner assignments. The red corner correlation exists because the UFC assigns corners based on ranking, championship status, and experience - the same underlying factors that drive fight outcomes.

Key distinctions:

- Betting odds reflect public perception, line movement, and bookmaker risk management
- The model analyzes actual performance metrics: strike accuracy trends, takedown defense patterns, cardio indicators, opponent-adjusted statistics,

etc.

- The value comes from divergence identification based on as much statistical information as possible. The corner bias is simply real world information that the model can learn to incorporate better than average bettors.

The unbalanced training preserves the signal that corner assignments carry meaningful information about fighter quality, information that's already baked into the real-world problem you're trying to solve. Throwing away that signal artificially handicaps the model's ability to make properly calibrated predictions.

What About Including the Odds?

This is a hotly debated topic in the sports betting community. Bill Benter, one of the fathers of algo betting, argued that the odds are best to be included because they encode so much information. Having been testing this hypothesis for many thousands of hours, my conclusion is that if you don't have a certain level of extremely high quality engineered features, then yes you should include the odds. But at a certain point, the features you engineers will actually encode more information in combination than the odds do. At that point, including the odds simply lowers the ROI.

Mathematical Mechanism

When you include odds as a feature, you're introducing a variable that represents the market's aggregated probability estimate:

$P_{\text{market}}(\text{fighter1_wins})$. This creates several mathematical problems:

1. Feature Dominance and Prediction Convergence

Tree-based models will heavily weight the odds feature because it exhibits high mutual information with the target across the entire dataset. The model's predicted probabilities become:

$$P_{\text{model}}(\text{fighter1_wins}) \approx \alpha \cdot P_{\text{market}} + (1-\alpha) \cdot P_{\text{stats}} \text{ where } \alpha > 0.5$$

This forces convergence toward market consensus.

2. Outcome Distribution Skew

Including odds biases the model toward predicting high-frequency, low-profit outcomes (favorites). Your engineered statistics without odds bias toward identifying low-frequency, high-profit outcomes (underdog value).

3. +EV Prediction Accuracy Inversion

The critical insight: ROI optimization requires accuracy specifically on profitable bets, not overall accuracy.

- **With odds included:** Model achieves ~74% accuracy but concentrates correct predictions on favorites (odds = 1.2-1.8, profit margin = 20-80%)
- **Without odds included:** Model achieves ~71% accuracy but concentrates correct predictions on underdogs (odds = 2.5-4.0, profit margin = 150-300%)

ROI Mathematics

Consider two hypothetical models:

- **Model A:** 75% accuracy, predicts favorites 85% of the time → Expected ROI $\approx -2\%$ (high accuracy, low margins, vig erosion)
- **Model B:** 50% accuracy, predicts underdogs 60% of the time → Expected ROI $\approx +15\%$ (lower accuracy, exponentially higher margins)

The fundamental equation:

$$ROI = \sum (accuracy_i \times frequency_i \times profit_margin_i)$$

Your empirical results demonstrate that excluding odds increases accuracy_underdog dramatically while slightly decreasing accuracy_overall. Since profit_margin_underdog $\gg$ profit_margin_favorite, the ROI optimization occurs through maximizing performance on the subset of predictions with the highest profit potential.

Information Compression Loss

Odds represent market consensus based on public information flow. Your engineered features capture orthogonal signals that specifically identify cases

where statistical analysis diverges from public perception—exactly the scenarios that generate +EV opportunities. Including odds suppresses these divergent signals in favor of consensus alignment, destroying the edge that creates profitable betting opportunities.

The Vanity Metrics Problem in Machine Learning

You will see many other novice machine learning engineers practice their skills against sports prediction. They will calibrate their models against vanity metrics like accuracy in most cases. They will see evaluations like 85% accuracy, then not have any motivation to figure out why their model will fail in the real world then make a YouTube or Medium post about it.

This represents the fundamental disconnect between academic machine learning and profitable real-world application. The sports prediction space is littered with impressive-sounding accuracy claims that evaporate when subjected to actual betting markets. These engineers optimize for metrics that sound impressive in blog posts rather than metrics that generate alpha.

The vanity metrics obsession creates a particularly insidious blind spot: data leakage and overfitting become invisible when you're chasing high accuracy numbers. Consider the classic example in YouTube tutorials (<https://www.youtube.com/watch?v=LkJpNLlaeVk>) where a model achieved impressive accuracy predicting UFC fights using Elo ratings... except they used post-fight Elo ratings for training, meaning the model literally saw fight outcomes during training. This is textbook data leakage: using future information to predict past events.

Data Leakage Examples in Sports Prediction:

- Training on post-game statistics to predict game outcomes
- Including betting line movements that occurred after the event
- Using season-end rankings to predict mid-season games
- Including opponent-adjusted metrics calculated after the fight

The overfitting trap compounds this problem. Hyperparameters are the configuration settings that control how a model learns: things like learning rate,

tree depth, regularization strength. I can easily tune these hyperparameters to achieve 78% accuracy on my validation set by optimizing for that specific data slice. But this creates a model that memorizes the validation set's quirks rather than learning generalizable patterns.

The feedback loop is broken: High accuracy on test data → immediate gratification → publish results → never discover real-world failure. There's no motivation to dig deeper because the vanity metric has been satisfied. The engineer never learns that their 85% accuracy model would lose money consistently because they never actually test it against betting markets.

Real-World Adversarial Markets

Real-world sports betting is an adversarial market where you're competing against:

- Professional odds compilers with decades of experience
- Sophisticated betting syndicates with proprietary data
- Market makers who adjust lines in real-time based on money flow
- The inherent vig that makes break-even betting a losing proposition

Simply achieving high accuracy on historical data means nothing if your predictions can't consistently identify mispriced markets. The difference between academic exercise and profitable application is the difference between predicting outcomes and finding edges.

Lessons from Four Years of Iteration

I'm four years into this space and I've made all the same mistakes. I've built models that achieved 78% accuracy and lost money consistently. I've spent months optimizing for log loss improvements that translated to worse ROI. I've fallen into every trap outlined above because the feedback loop between model performance and real-world profitability is opaque until you actually start tracking betting results over extended periods.

This is a far more complex problem than I ever initially thought 4 years ago, but I believe what's outlined above is one of the reasons I've been seeing almost 20% ROI over the past few years in my free, public predictions. The key insights

around dataset balancing, odds inclusion, and ROI-focused optimization came from years of iterative failure and debugging, not from following standard ML tutorials.

The engineering rigor required to build profitable models demands treating accuracy as a vanity metric and ROI as the only metric that matters with the knowledge that betting markets shift as time goes on and you must constantly retrain the model to match the current zeitgeist. Hence why I'm on version 6.3 right now. I've redone 100% of the code 6 times now. If you want to avoid the same mistakes I've made, feel free to reach out. I'm an open book on exactly how I built this model, no secrets. And also, as usual, shoutout to Chris from Wolftickets.ai for being one of the very very few people in this ML for sports prediction field that actually shared his technical knowledge and saved me endless hours of wasting my time. I like paying that forward to others who are interested in this space.

Why Don't You Recommend The Positive EV Picks?

April 19, 2025

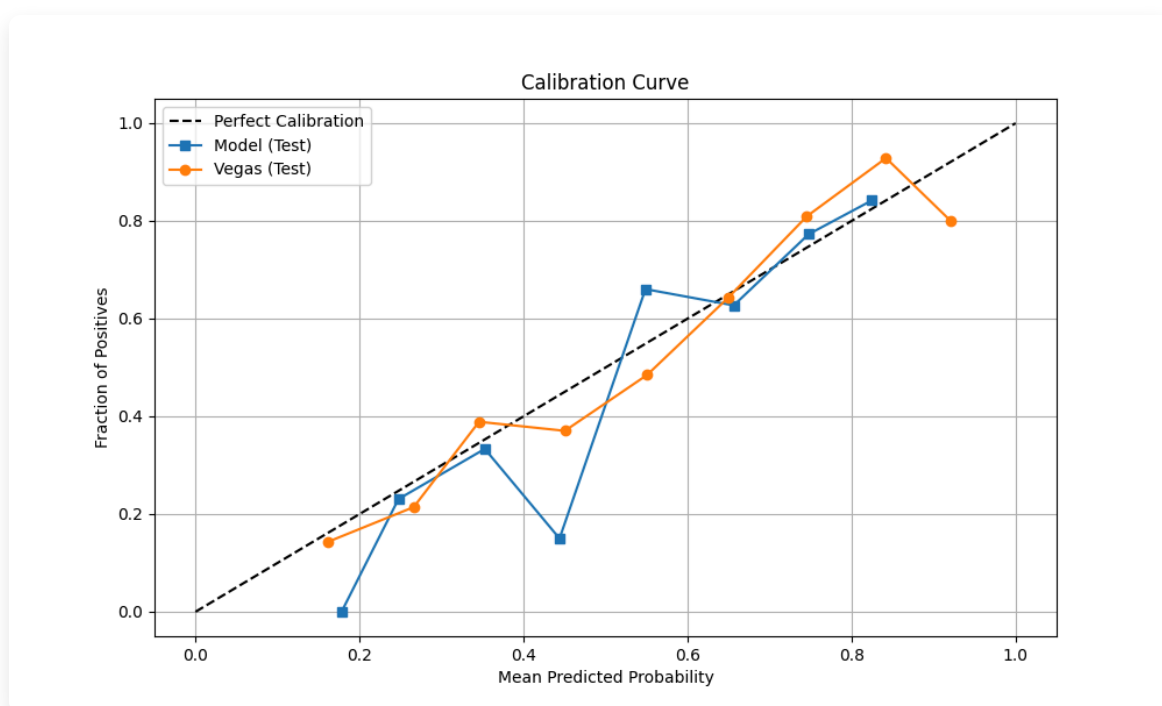
I have released v6.1 of the model. Large improvement in accuracy but most specifically in ROI. So let's talk betting strategy. One question I get asked practically every day is, "why do you recommend only the AI picked winners even if they're not +EV?"

The best tech available for predicting UFC are binary classification algorithms. These are algorithms specialized in classifying a fighter as either 1 or 0, win or loss. The confidence score that the algorithm comes up with is not its fundamental strength.

"So why don't you calibrate it?"

Because the calibration tech we have for these algorithms suck. Platt Scaling and Isotonic Regression are the two most potent weapons we have to calibrate these predictions. I have experimented a lot with them. 100% of the time they suck. They lower the accuracy and the ROI. I've experimented with custom calibrations but it's just too inconsistent and I don't see a promising way forward. I would absolutely love to have better calibration, so if you have any ideas please let me know. That being said, the ultimate goal here is risk-adjusted returns. Calibration is a tool to help us get there but it's not necessary to acheiving the goal.

Anyway, who cares about calibration if we're making fat ROI? Take off your sports bettor hat and all the "fundamentals" you know like +EV is all that matters to be successful. Put on your machine learning cap and look at the data. This is the calibration curve:



Note how the model is highly underconfident in its 50-65% confidence score picks and highly overconfident in it's 35-50% confidence scores. The nature of the algorithm is to maximize it's ability to successfully pick the winner and it does this at the expense of accurate calibration. It clusters it's confidence scores in the 50-65% range. AND THAT'S OK!

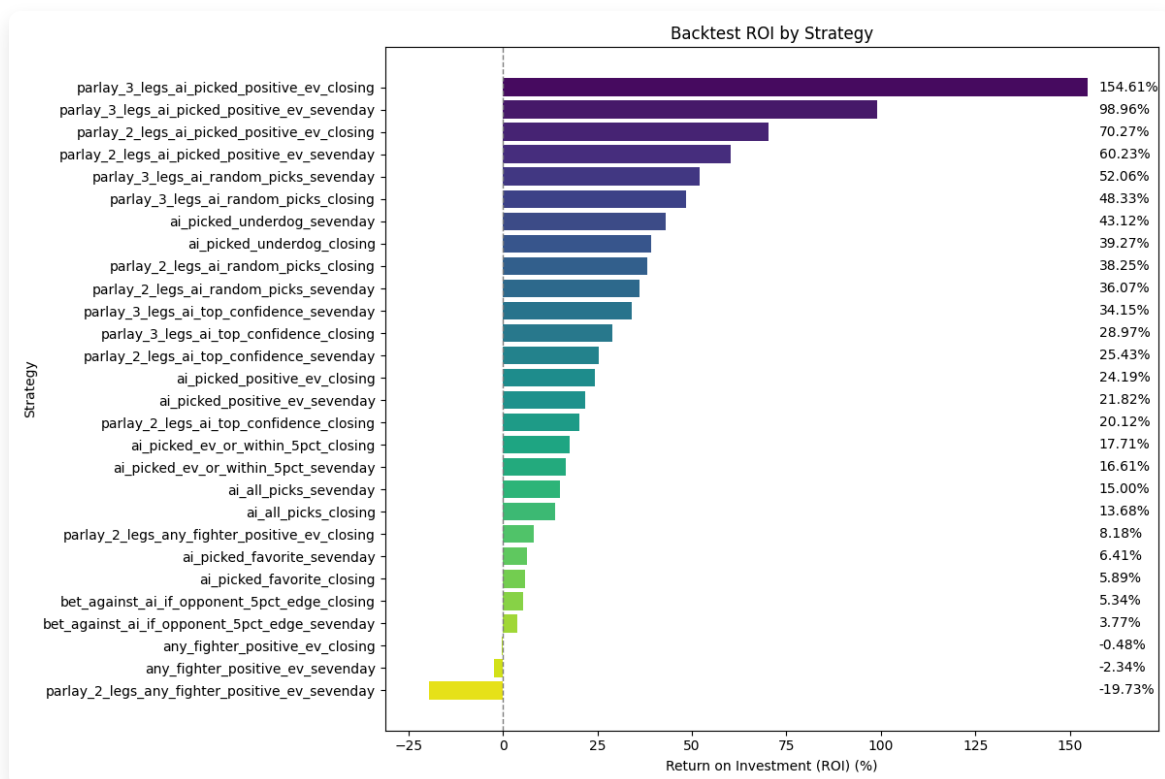
Here's the model evaluations and vegas evaluations on the last year of unseen fights:



Performance Metrics Comparison

Metric	Vegas Odds	MMA AI Model
Accuracy	0.690	0.710
Precision	0.725	0.697
Recall	0.725	0.859
F1 Score	0.725	0.770
Log Loss	0.587	0.602
Brier Score	0.201	0.207

Note the model is more accurate than Vegas, but it's logloss and Brier scores (essentially real world win%) are worse. Again, THIS IS OK. Why? Because look at these profit strategy backtests over the last year of unseen fights:



The fundamental strategy of betting on 100% of the AI picks on the closing odds is 14% profitable (ai_all_picks_closing). The strategy of picking only the

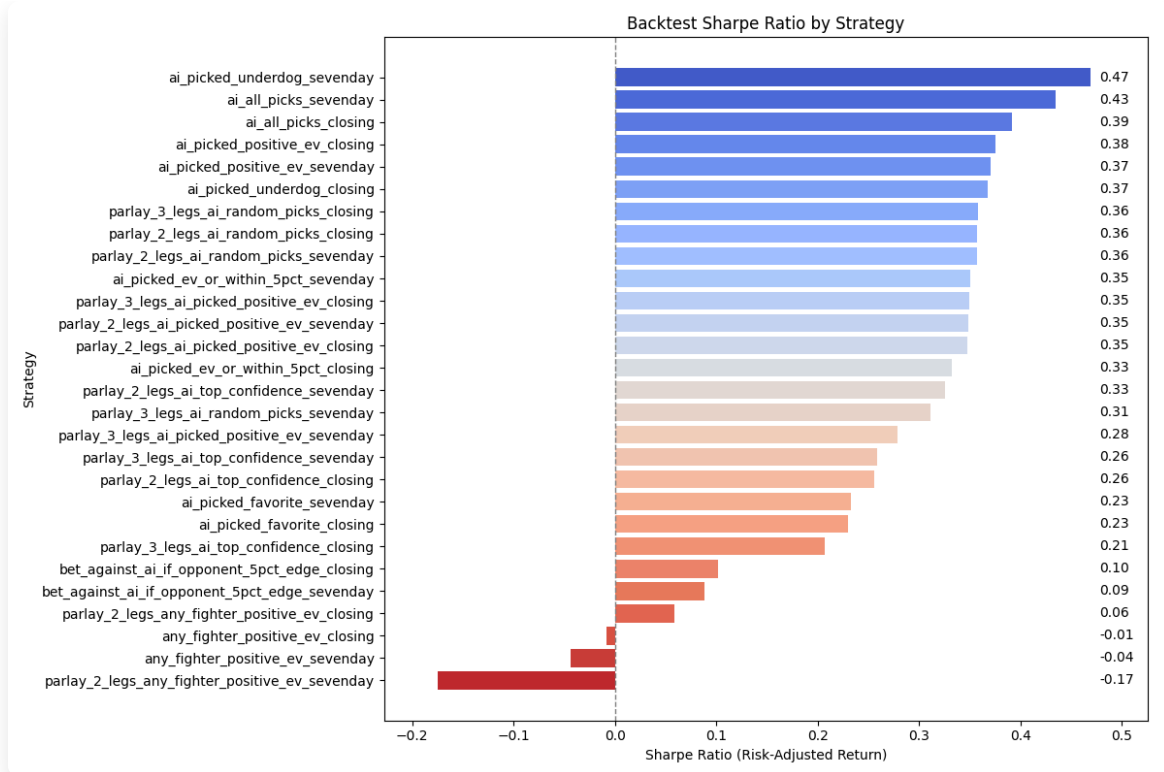
+EV fighter based on the AI confidence score whether the AI picked that fighter to win or not is -0.5% unprofitable (any_fighter_positive_ev_closing).

Interpreting The Strategies

None of this is to say that +EV isn't a helpful metric. Look at another fundamental strategy, betting only on +EV AI-picked fighters (ai_picked_positive_ev_closing): +24.2% ROI. 10% higher than just doing down the line bets but it means sometimes there'll be events where you don't get to bet which makes me sad.

So let's put this all together. Why are parlays so good with this machine learning algorithm? It's because the algorithm is so gosh darn good at picking winners. Parlaying the AI picks, whether +EV or not, is a highly profitable strategy. The best strategy is doing 2 to 3 leg parlays on the +EV AI picked fighters. I'll probably move into posting these parlays again in the homepage.

EDIT: Shoutout to [@Heisenb3rg](#) on X for bringing Sharpe Ratios to my attention! This is a crucial metric from finance that measures risk-adjusted returns. Essentially, it tells us how much return we're getting for the amount of risk (volatility) we're taking on with a particular betting strategy. A higher Sharpe Ratio is better.



Looking at the Sharpe Ratio analysis, we can see how different strategies compare not just on raw ROI, but also on how bumpy the ride is. For v6.1, the 'ai_picked_positive_ev_closing' strategy seems to offer a strong balance, providing good returns without excessive risk compared to some others. However, the 'ai_all_picks_closing' still performs well from a risk-adjusted perspective, offering broader betting opportunities.

So, what's the takeaway? While different model versions might favor underdogs or favorites differently, the data consistently points towards a simple, robust approach. Considering both the ROI and the Sharpe Ratio (risk-adjusted return), the strategy of betting on **all AI picks using the odds available 7 days before the event** (ai_all_picks_7day) emerges as the winner. It offers the best blend of profitability and lower volatility, making it the simplest and most effective core strategy based on the current analysis.

It's important to remember that model performance, especially regarding favorites versus underdogs, can shift between versions and timelines. Therefore, relying solely on specific odds ranges like underdogs only might be less reliable long-term despite being very profitable over the past year. Instead, I think focusing on fundamental strategies evaluated through metrics like ROI and Sharpe Ratio is key. For the current model, based on the patterns I've seen over

the years, I think I'm going to stick with the core strategy ai_all_picks_sevenday which provides the best risk-adjusted performance indicated by the Sharpe Ratio. To get higher returns at higher risk, 2 to 3 leg parlays of your choice are a good option.

MMA-AI UFC 314 Predictions

April 12, 2025

Here are the AI predictions for UFC 314.

AI Predictions - UFC 314



AI Predictions and Analysis for UFC: Emmett vs Murphy

April 4, 2025

Check out the latest video analysis and AI predictions for the upcoming UFC matchup between Emmett and Murphy. The AI generated interpretation of my face always cracks me up.

AI Predictions for UFC Emmett vs Murphy - MMA-AI.net



New Video: Understanding MMA-AI Predictions

March 19, 2025

Check out our latest video explaining how MMA-AI makes its predictions and why our model has been consistently outperforming Vegas odds.

UFC Fight Night: Moreno vs Erceg - AI mathematical predictio...



Demystifying the MMA-AI Prediction Algorithm: How We Predict UFC Fights

March 18, 2025

After countless requests from the community, I'm finally pulling back the curtain on how our UFC fight prediction algorithm works. This isn't just another black-box system—it's a carefully engineered, multi-level approach to fight analysis that's consistently outperforming Vegas odds.

Why Share Our Secret Sauce?

Many have asked why I'd reveal the inner workings of a profitable system. The truth is simple: I don't believe in keeping this knowledge secret. Much of this algorithm was developed with the assistance of AI tools that generated about 90% of my code. As someone who's been an open source developer for 15 years, I firmly believe that open knowledge sharing accelerates human progress. I don't have a moat to protect—and frankly, the more people who understand these concepts, the faster our collective knowledge will advance. Sports prediction is hard enough that even with this blueprint, you'll still need to invest thousands of hours to truly master it.

The Four Levels of Fight Prediction

Level 1: Foundation Stats + Bayesian Gamma-Poisson Smoothing

We start with raw fight statistics—strikes thrown, takedowns landed, control time, and dozens more. But raw numbers can be misleading, especially for rare events like submissions. This is where Bayesian Gamma-Poisson smoothing comes in.

Imagine a fighter who's attempted only one submission in their career and landed it. Is their submission accuracy really 100%? Probably not. Gamma-Poisson smoothing helps us balance observed data with prior knowledge, preventing outliers from skewing predictions. For high-volume stats like significant strikes, the smoothing effect is minimal. But for rare events like submissions or knockdowns, it provides crucial stability by "pulling" extreme values toward more realistic expectations based on the fighter's division and overall UFC averages.

Level 2: Comparative Analysis

Next, we calculate a fighter's efficiency metrics: accuracy (how often attacks land), defense (how often they avoid opponents' attacks), output per minute, and the ratios between fighters across all statistics. This gives us a clearer picture of how fighters perform relative to each other, beyond just counting actions. A fighter might land fewer strikes overall but have significantly higher accuracy—a crucial distinction our model captures.

Level 3: Time-Weighted Averages + Variability

MMA evolves rapidly, and a fighter today isn't the same as they were three years ago. We calculate both standard and time-decayed averages with a 1.5-year half-life for all statistics. This means a fight from three months ago has far more impact on our predictions than one from three years ago.

Additionally, we measure the variability of these stats using Median Absolute Deviation (MAD) instead of standard deviation. MAD is less affected by extreme outliers, providing a more stable measure of how consistent a fighter's performance has been.

Level 4: Adjusted Performance (AdjPerf)

Our most sophisticated metric answers a critical question: how does a fighter perform against a specific opponent compared to how that opponent's previous adversaries performed? This z-score normalization looks like:

$$\text{stat_adjperf} = (\text{fighter1_stat} - \text{fighter2_stat_prev_opp_avg}) / \text{fighter2_}$$

In plain English: we're measuring how much better or worse a fighter performed against their opponent compared to what we'd expect based on the opponent's history. If a fighter lands more strikes against someone who's historically difficult to hit, that's far more impressive than landing the same number against someone who's easily hit. AdjPerf captures this crucial context.

Building the Prediction Model

With thousands of engineered features from these four levels, we use Autogluon (thanks to Chris from Woltickets.ai for this recommendation) to train our prediction model. Using presets like "experimental" and time-ordered data splitting (typically 80/20 or 90/10 train/test), we run extensive cross-validation to ensure reliability.

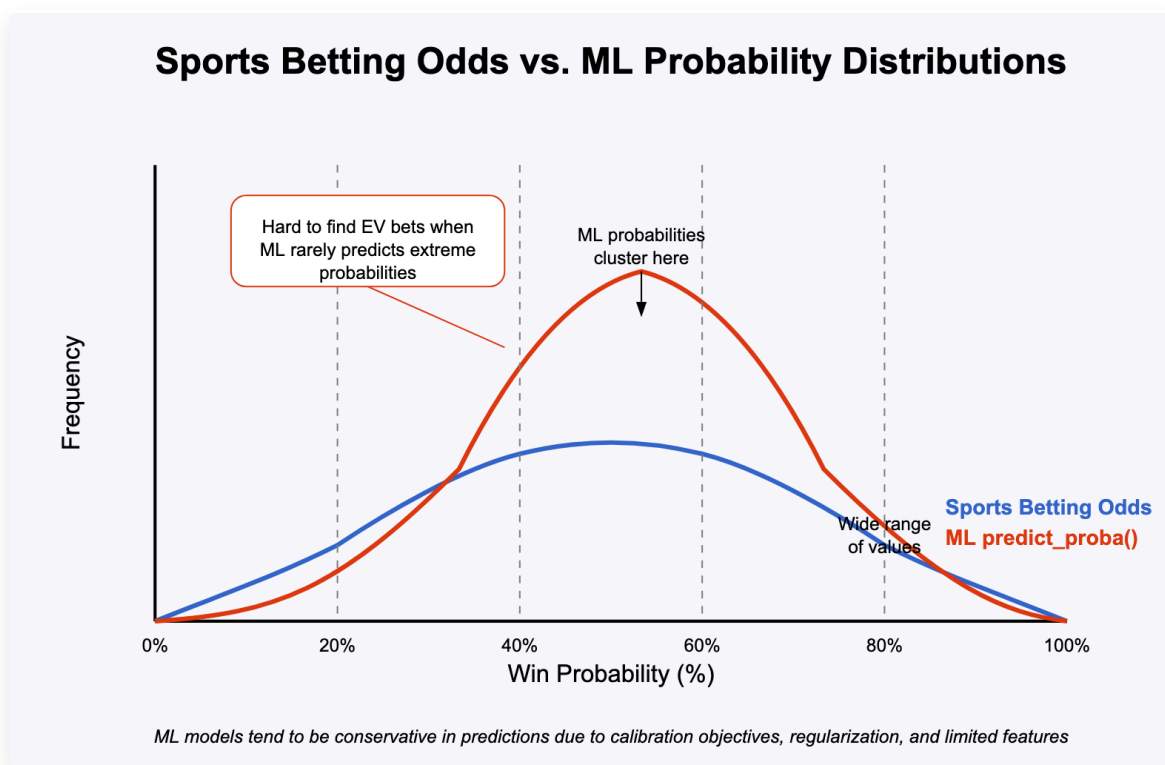
There's no definitive guide to sports prediction—we've conducted thousands of hours of testing to find what consistently beats Vegas odds. Everything from feature selection to hyperparameter tuning requires constant experimentation and refinement. What works for NFL might not work for UFC, and what worked last year might not work next year.

The final model combines about the 30 best of these features, weighted according to their predictive power. The result is a system that consistently identifies value bets where our predicted win probability exceeds what Vegas odds imply.

The Practical Limitations of ML Probabilities

One of the humbling lessons I've learned over the years is about the relationship between machine learning and betting strategy. Despite the sophisticated nature of our model, I've had to accept that ML algorithms have inherent limitations when it comes to probability calibration. While traditional betting approaches rely heavily on expected value (EV) calculations, I've discovered that applying these same principles directly to ML outputs can be problematic. Our tabular ML models excel at binary classification—essentially determining which fighter is more likely to win—but their confidence scores aren't necessarily true probabilities in the statistical sense. Even when optimizing for log loss (which theoretically improves probability calibration), there remain subtle biases and distortions in how these models estimate probabilities.

Through trial and error, I've found that treating the model's outputs as relative confidence levels rather than exact win probabilities leads to more consistent results. Instead of rigidly applying EV formulas, we use confidence thresholds as a filtering mechanism to identify promising bets. This pragmatic approach acknowledges the model's strengths in pattern recognition while respecting its limitations in precise probability estimation—a balance that has proven more effective than assuming perfect calibration.



Visualization of our model's prediction distribution compared to actual outcomes, showing the effectiveness of our approach despite calibration challenges.

Why Current LLM-Based Fight Predictions Fall Short

I've extensively tested Large Language Models (LLMs) like GPT-4 for fight predictions, and the results have been consistently disappointing. The fundamental limitation is clear: today's LLMs lack the ability to analyze fight footage. They're trained primarily on text data, which means they miss crucial visual information—a fighter's movement patterns, subtle defensive vulnerabilities, changes in stance, or signs of fatigue that only appear on video. Statistical data can tell us a lot, but the visual dimension of fighting contains irreplaceable insights that no spreadsheet can capture. The good news? This limitation is temporary. Within the next 1-2 years, we'll see multimodal AI

systems trained on vast libraries of video footage, capable of analyzing thousands of fights frame-by-frame. When that happens, AI fight prediction will undergo a revolutionary leap forward. Until then, the most effective approach remains combining sophisticated statistical modeling with human expertise for context and interpretation.

Conclusion: The Never-Ending Journey

MMA prediction remains as much art as science. While our technical approach provides an edge, the sport constantly evolves, and so must our model. Every event brings new data, every fighter brings new patterns, and our system continually adapts to these changes.

Whether you're a casual fan or a serious bettor, I hope this behind-the-scenes look helps you appreciate the depth of analysis that goes into each prediction you see on MMA-AI.net. And if you're inspired to build your own model? Even better—innovation thrives when knowledge is shared.

Model v6 Release

March 15, 2025

Model v6 is available for tonight's event and events going forward. This is intended to be a long-term release.

Changes:

- **Unit tests for everything**
 - Finally we have unit tests for all calculations so I can sleep at night. At least now if the model fails I know it's not some unknown bug.
- **Bayesian Gamma-Poisson**

- We switched out from just doing Bayesian Beta Binomial smoothing in accuracy/def and per_min calculations to a Bayesian Gamma-Poisson for all stats starting at the very beginning. This is a more robust and consistent smoothing factor.
- **Swapped stddev for median absolute deviation for the zscore normalization stat adjperf**
 - Adjperf layer of feature engineering changed from:
$$\text{stat_adjperf} = (\text{fighter1_stat} - \text{fighter2_stat_prev_opponents_avg}) / \text{fighter2_stat_prev_opponents_stddev}$$
to:
$$\text{stat_adjperf} = (\text{fighter1_stat} - \text{fighter2_stat_prev_opponents_avg}) / \text{fighter2_stat_prev_opponents_mad}$$
 - Median absolute deviation (MAD) is a more robust measure of deviation for sports statistics because it's not as vulnerable to outliers.

Future:

Have a few stats I'd still like to engineer especially around the win conditions of submissions vs KO vs decision.

I have also added visualizations to the predictions. Just hover over the fight or click on the fight to see how each fighter compares to each other using a normalized stat difference. Note that these visualizations do not account for how the model is weighing each stats, it's just the normalized difference in the stats between the fighters to give you an idea of where each fighter has an advantage. Larger surface area = better stats but you may notice sometimes the fighter with the larger surface area is not picked, this is due to the fact the stats are not weighed equally by the model and also because the model makes some complex relationships between the stats that won't be reflected by the visualization.

Unit Testing Not Done Yet

March 1, 2025

Based on the past 2 months of performance I'm fairly convinced I have some bugs in the nonunit tested code. I've started the unit testing but I'm not done yet so take this event (3-1-2025 UFC Fight Night) with a grain of salt, might be a good one to skip for now, but I should have lots of time after work to finish unit testing next week and I'll post an update about bugs found once I'm done.

v5.2 Model Release

February 22, 2025

After countless hours of trial and error testing this week, we're implementing a few key refinements to the model. Rather than making sweeping changes, we're focusing on targeted improvements that have shown consistent benefits in our testing:

1. Updated Training Data Window

We're moving the cutoff date for training data from 2014 to 2016. While I've been using 2014 as the starting point for the past 4 years, it's time to shift forward to better capture recent trends in the sport. MMA evolves rapidly, and data from 10 years ago may not be as relevant to today's fighting landscape as it once was.

2. New Feature: UFC Age

We've added a simple but effective new stat: UFCAge. This measures the amount of time a fighter has been in the UFC. While straightforward, this metric helps capture valuable information about a fighter's experience at the highest level of competition.

3. Enhanced Feature Layering

Previously, we were using two primary layers of features:

- Decayed adjusted performance (dec_adjperf_dec_avg)
- Opponents' decayed average (opp_dec_avg)

Now we're adding a third layer: the fighter's individual stats (like strikes_landed_ratio_dec_avg). This addition has proven valuable - by combining all three layers of a key base stat (like head_landed_ratio), the algorithm seems better equipped to predict how each fighter will perform relative to the broader UFC population.

Looking Forward

I'm preparing to start producing analysis videos to help explain the model's decision-making process and break down specific fight predictions. The site has been updated with the latest calibration curve, model evaluations, and feature importance rankings to reflect these changes.

As always, these updates are focused on one goal: improving our ability to predict fight outcomes accurately. The changes might seem subtle, but in the world of fight prediction, small edges often make the difference.

Adjusting for Adjusted Performance and Sneaky Bug Fixes

February 8, 2025

Goddamn that took forever. There was a bug in the zscore normalization stat I named _adjperf. If you recall, the calculation for adjperf is:

```
stat_adjperf = (fighter1_stat - fighter2_prev_fight_stat_opp_avg) / figl
```

- f2_stat_opp_avg is the historical opponent average against fighter2 pulled from fighter2's previous fight.
- f2_stat_opp_sdev is the historical opponent standard deviation pulled from fighter2's previous fight.

This stat is extremely complicated to implement and was the reason we were predicting so few fights ever since v5 was released with it included. We train with only data known pre-fight so we shift all the data backwards by 1 fight for all fighters. The consequences:

Case #1:

All fights where at least one fighter is debuting are immediately eliminated because their data is 100% null after we shift the data backwards by 1 fight and we can make no comparisons between them and their opponent.

Case #2:

All fights where at least one fighter has only 1 previous fight is useless because one-previous-fight fighter has a 0 in fighter2_prev_fight_stat_opp_sdev for their opponent's adjperf calculation.

Case #3

One fighter has 3 previous fights, the other fighter has 2. Again, adjusted_performance for fighter1 will be 0.

And so on. It means that _adjperf is causing an enormous amount of fights to be dropped.

I've tried to fix this by calculating an initial standard deviation for all first time fighters. I collect all stats of first-time fighters on a per weightclass, per stat basis. So your _adjperf score against first-time fighters is based on how good or bad you did compared to the median first time fighter's stats. This feels fine,

because we're dropping all fights where either fighter only has 2 previous fights anyway.

Bug Fixes

So there was a fun little bug affecting ~2% of the training data where fighter2_stat_opp_sdev would end up being 0.00000002 or something so the adjperf score would end up being like 11,839,902 instead of a more normal 1.5 or -1 (i.e., fighter1 scored 1.5 standard deviations above what fighter2 usually allows in that stat from their historical opponents). The reason was because I only use time-decayed averages in the final model to capture recent trends in both fighters and their opponents and the time-decayed standard deviation calculation is a bit complex:

```
weight = EXP(-λ * ((T - t) / 365.25))
```

Soooooometimes, $T = 7.777777777$ and $t = 77.77777778$ due to rounding in Postgres double precision. I have fixed this by capping the minimum standard deviation of stats to their 5th percentile. This is to avoid insane standard deviations in tiny edge cases like Khamzat Chimaev's early UFC run where his opponents only scored like 4 strikes against him in 4 fights then he fought Gilbert Burns who hit him 223 times. This meant Burns' `_adjperf` score was insane because he took Khamzat like 400 standard deviations away from his previous opponent averages.

Last, I had some small bugs affecting the filtering of training data. The order of filtering the data matters a LOT and I was doing some dumb stuff like filtering out Draws and stuff, THEN calculating number of fights for each fighter and filtering on number of fights which is wrong because I already filtered out fights before calculating number of fights for each fighter.

OK! That about wraps up the updates this week. It took a really long time to get through all those bugs so I'm trying to get the updated predictions out before this event, bear with me.

Updates to Betting Strategy

January 18, 2025

First, we're changing the wording from "AI win%" to "AI confidence". This is to sort out the confusion that many traditional bettors have. This algo is not specialized in finding +EV. It is a binary classification algorithm, meaning it is very good at figuring out who will win, but not the real world win % chance. `Predictor.predict(upcoming_fights)` returns a binary value of 1 = fighter1 predicted to win, 0 = fighter1 predicted to lose. We can do `predictor.predict_proba(upcoming_fights)` and get it's best guess at each fighter's implied probability but generally this probability is not well-calibrated because this is specifically a binary classifier.

Second, the reason we're doing parlays is because they're essentially a multiplier of the profit from the single picks. You can risk less units to win more. You don't have to do this, you can just do the straight picks and limit the boom and bust cycle of the parlay strategy, but backtesting is showing the parlay strategy consistently outperforms the straight picks in ROI.

On this note, up until now I've been randomly creating the parlays. I've been trying to figure out how to get rid of that randomization. Sorting the fights by AI confidence or inconfidence ended up performing worse than random picks. So I thought about what does the algorithm crave to become more accurate? Data.

The new parlay strategy is based on backtesting showing more consistent results by sorting the parlays by total combined fights both fighters have had. Islam has had 15 fights in the UFC. Renato Moicano has had 17 fights. That's 32 fights of data that the algorithm has had time to calibrate on. So whatever the algorithm picks on this fight is likely to have the most consistent accuracy. That's why we're seeing islam in 3 parlays this event.

This strategy could use more refinement, and ideally we'd actually just create every possible 3 leg parlay and set that up as the predictions for every event.

The only reason I don't do that is because it takes forever to input all those parlays into betmma.tips and the bookie but maybe once I automate the placing of pre-event picks into betmma.tips or something then I'll start doing all parlay combinations possible as the recommended picks.

New Key Feature: Adjusted Performance

January 11, 2025

Alright, so I'm finally ready to talk about my new key feature: Adjusted Performance. This has been one of those "why didn't I think of this sooner" moments, but also "holy crap this is a nightmare to implement." Let me give you a quick taste of what it looks like mathematically:

```
fighter1_stat_adjperf = (fighter1_stat - fighter2_stat_opp_avg) / fighter2_stat_opp_avg
```

We'll call this `f1_stat_adjperf` for short. The big idea is to measure how much a fighter's performance in any given fight exceeded or fell short of what we'd expect their opponent to allow. If you go in there against some unstoppable jab machine who normally forces everyone to eat 50 jabs per round, but you manage to hold them to only 20, well, that's huge. But if you never do the math to figure out what your opponent's "baseline" is, you'll just record "Fighter1 absorbed 20 jabs" and maybe think it's not that great. Meanwhile, that's incredibly good compared to the 50 jabs everyone else took. Hence, adjusted performance.

Understanding the `_opp` Suffix

Before going any further, let's talk about the `_opp` suffix. First, `_opp` is the post-fight stats your opponent did against you. So if you see something like

f2_stat_opp_avg, that means the stat is referencing "what your opponent's opponents did against them." For example:

- f1_stat_opp_avg: The average of your opponents' performances against you in all their fights.
- f2_stat_opp_avg: The average of your opponent's opponents' performances against your opponent in all their previous fights.
- f2_stat_opp_sdev: The standard deviation of your opponent's opponents performance in those same fights, giving us a measure of the volatility or variability in their performance.

So, if your opponent has 40 strikes landed against them then f2_strikes_opp_avg is 40.

This means, to measure your adjusted performance, you compare your post-fight stat in the fight to their pre-fight _opp_avg, and then scale it by pre-fight _opp_sdev. If you get 60 strikes against someone who averages getting hit 30 times, you're 1 standard deviation above average against that person.

Why It's So Valuable

Let's be honest: raw stats can lie, or at least mislead. If a fighter's output is just "I landed 30 strikes," that doesn't tell me how many strikes they should have been able to land. If they were fighting an iron-clad defensive wizard who typically only allows 10 strikes, then landing 30 is insane. On the flip side, if your opponent is basically an open punching bag who gives up 60 strikes on average, then landing 30 is actually pretty weak.

Adjusted performance changes the game: it says, "30 strikes might be good or bad, but let's see how it compares to what your opponent usually allows." Then, for even more nuance, it's scaled by the opponent's historical standard deviation—so you don't artificially inflate your stats just because you faced someone with wide variability on a certain stat.

The Complexity: Where Do You Even Get These Numbers?

The problem with pulling off something like f1_stat_adjperf is that you actually have to calculate your opponent's _opp_avg and _opp_sdev. In other words:

- Grab all your opponent's previous fights.
- For each of those fights, figure out the stats they allowed to their opponents.
- From that, compute the average allowed stats and the standard deviation of those stats.
- Then bring that back into your current fight to see how well or poorly you did.

This is where the data pipeline can get insane, because you might have a fighter who has 15 or 20 fights, each with a different set of opponents. Some of those opponents have 30 fights apiece. Doing this for every fighter means you need to traverse a huge web of fight stats.

If you run a naive approach—like just using Panda's groupby and merges all day—it becomes unbelievably slow at scale. This is why I had to do some major refactoring, rewriting a chunk of my data pipeline to pull from a properly indexed database (Postgres, in my case). Once your data is properly structured, it's much faster to do these calculations in a single pass or via specialized queries, rather than stumbling around in memory merges.

Time-Decayed Averages & Time-Decayed StdDev

But wait—there's more. I decided to do a time-decayed average (and corresponding standard deviation) with a 1.5-year half-life. That means that a fight 3 months ago is given a lot more weight than a fight 5 years ago, which is basically ancient history in fight years.

Now the complexity is multiplied by, like, a factor of 10. Because to get `f2_stat_opp_avg`, I can't just average everything your opponent has done; I have to:

- Grab each fight's stat.
- Weight it exponentially based on how recently it happened.
- Sum up those weighted stats.
- Divide by the sum of the weights.
- Then do it all over again for the standard deviation.

And let's not forget: we do this for every single fight, across thousands of fights, across hundreds of fighters. That's why I always say data engineering is half the battle.

Why Bother?

So why go through this code-wrangling fiasco when your standard raw stats might be "good enough?" Because fights are context-dependent. If you have a stand-up specialist with insane takedown defense, but no one has tested it in years, your raw stats might not reflect the real picture of how they handle a brand-new style. By blending in adjusted performance stats, you're no longer stuck just describing how many strikes or takedowns a fighter landed; you're describing how well they did compared to what their opponent typically experiences—and you're discounting or boosting older fights according to their recency.

This is how we start to see nuanced differences that no raw stat alone can show. It's the difference between "Fighter A landed 40 strikes" vs. "Fighter A forced a 1.5 standard-deviation drop in Fighter B's typical striking output." That second statement captures so much more power. If you can integrate these insights into your model, you get a far more realistic prediction of how a matchup might turn out.

MMA-AI.net v5 New Years Updates

December 27, 2024

v5 beta is *might* be done in time for the next event. I'm 100's of hours in. The data processing and model training is basically done, I just need to figure out the final feature set and do some tuning. Then I need to write the future prediction code for creating and cleaning the data of future fights. Finally, we're beating Vegas accuracy/log loss without including the odds or rating/ranking stats like an Elo score. That's not to say I won't include those in the future, but it's a great sign that the fundamentals of the model are improving. I would like to

thank my \$200/mo ChatGPT o1 Pro subscription for making this possible. That model absolutely rules at complex code, especially statistics and math.

Here's a recent training run to give you an idea of current levels of performance. This isn't the final model performance, it's just from tinkering with the feature set and training parameters:

```
Model Performance:  
Training log loss: -0.611  
Validation log loss: -0.575  
Test Log loss: -0.612  
Training accuracy: 0.671  
Validation accuracy: 0.698  
...
```

Test accuracy: 0.691

Basically, we're seeing somewhere around 68% to 69% accuracy on last year's fights (Vegas is currently about 67% accurate) that the model has never seen before with excellent log loss. Log loss is the ability of the model to accurately predict the chance a fighter will win. For example, if in its training data it predicts 50 fighters to have a 70% chance to win, and those fighters won 68% of the time, then the log loss is nicely calibrated. This is what allows us to check the EV of AI predictions versus Vegas odds.

Changes:

- Total rewrite
 - Switched from just pandas dataframes (SLOW) to Postgres SQL database (FAST)
 - Greatly improved code readability, design, and efficiency for easier future updates
- Features
 - Total feature overhaul
 - Per minute
 - Uses Bayesian Posterior Mean to smooth the outliers, zeros, and noise reduction

- Accuracy/defense
 - Uses Bayesian smoothing with a Beta prior to smooth the outliers, zeros, and noise reduction
 - Priors are calculated on a historical pre-fight per-weightclass, per-stat basis
- Ratio
 - Now does bounded ratio to prevent division by zero and outlier ratios
- Average
 - Now includes time decayed average rather than recent average

The reason we do all this smoothing is because when a fighter attempts 0 submissions and lands 0 submissions, giving them 0% accuracy is an unrealistic measure of what their submission accuracy would've been had they attempted any submissions. With smoothing it looks more like this:

$\text{Subs_land} / \text{subs_attempted} = \text{accuracy}$

$0 / 0 = 20\% \text{ acc}$ (similar to historical weightclass average submission accuracy)

$0 / 1 = 18\% \text{ acc}$

$0 / 10 = 2\% \text{ acc}$

These are just examples, not the actual numbers but you get the idea. It punishes fighters who never land any attempts but doesn't overpunish them which means future stats that are layered on top will be more realistic to their actual performance based on sparse data.

Second, no more recent averages over an arbitrary number of fights or dates. Time decayed averages are where it's at. Set a half life, like 1 year. Fights within this 1 year account for, let's say, 50% of the time decayed average. Fights the year prior account for 25% of the time decayed average, etc. This gives a much more precise measure of the fighter as he stands today rather than letting how he stood 5 years ago affect his current stats.

All of these changes above were nice and showed a moderate increase in the model's reliability. However, the real coup de grace was a final layer of statistical analysis over the stats that solved the following problem: if you fought nothing

but cans 20 fights in a row your stats would make you look better than Jon Jones. But if you fought nothing but top flight competition 20 fights in a row, your stats would look average to below average despite the fact you'd crush the can crusher. The main way people have solved this has been ranking or rating systems like Elo scores. While this is pretty effective it doesn't solve the core problem of the fighter's individual fight stats being a kind of isolated measurement of the fighter that lacks perspective. How do you turn these core, fight by fight stats into an interconnected web where each individual fight's stats can inform and affect the stats of other related fights?

I'll go into detail about the exact math I did to solve that problem along with the mathematical implementation of the Bayesian smoothing on my Patreon for subscribers here soon. See you next event, hopefully!

The Art of Not Sucking at AI: A Post-Mortem of My Model's Spectacular Face-Plant

March 24, 2024

After watching UFC 309 systematically demolish my model's predictions with the precision of a prime Anderson Silva, I've spent the past week in a caffeine-fueled debugging frenzy. Between muttering obscenities at my terminal and questioning my life choices, I've had some genuine epiphanies about AI development that might save others from my particular flavor of statistical hell.

The Problem with Yes-Man AI

Large Language Models are like that friend who encourages your 3 AM business ideas. "A blockchain-based dating app for cats? Brilliant!" This becomes particularly

dangerous when you're knee-deep in feature engineering and looking for validation rather than criticism.

After extensive testing, I've discovered something fascinating about GPT o1's mathematical capabilities. While most LLMs give basic statistical approaches, GPT o1 can dive deep into complex statistical problems. But the real breakthrough came from building an AI feedback loop: get statistical approaches from GPT o1, feed them to Claude for implementation (it writes cleaner code), then feed Claude's code back to GPT o1 for validation.

Even debugging has improved. When Claude's code throws an exception, feeding the error back works once. But for persistent issues, asking Claude "what do you need to debug this error?" is far more effective. It responds with diagnostic code that, once fed with real data, leads to actual fixes rather than band-aids.

This iterative process, combined with extensive prompt engineering and lots of sample data to help GPT o1 truly understand the problem domain, has led to the first major mathematical breakthrough in V5's development: our new Bayesian approach to handling fight statistics.

Bayesian Beta Binomial: The Zero Division Solution

This is only one of many many improvements in V5 but I find it super interesting so I'm writing about it and fuck you if you don't want to hear about it. Let's dive deep into how we handle the dreaded divide-by-zero problem in fight statistics. When calculating success rates (like submission accuracy or strike accuracy), we use Bayesian Beta Binomial analysis to provide meaningful priors that smoothly handle edge cases.

The approach works like this: Instead of naive division that breaks on zeros, we model each ratio as a Beta distribution where:

- α (alpha) represents successful attempts plus prior successes
- β (beta) represents failed attempts plus prior failures
- The posterior mean ($\alpha / (\alpha + \beta)$) gives us our smoothed estimate

For example, with submission accuracy:

$$\text{submission_accuracy} = (\text{submissions} + \alpha) / (\text{submission_attempts} + \alpha + \beta)$$

We determine our priors (α , β) through empirical Bayes by analyzing the historical distribution of success rates across all fighters. These priors vary by stat type, reflecting the different base rates we see in MMA:

- Submissions: Lower α and higher β values reflecting their relative rarity
- Strikes: More balanced α and β values reflecting higher occurrence rates
- Takedowns: Intermediate values based on historical success rates

This approach elegantly handles three critical cases:

- Zero attempts: Returns the prior mean ($\alpha/(\alpha+\beta)$)
- Small sample sizes: Heavily weights the prior
- Large sample sizes: Converges to the empirical rate

To understand why this matters, consider how submission accuracy was traditionally handled: A fighter attempting 10 submissions and landing none would be assigned 0% accuracy. This creates two problems: it skews averages downward, and when comparing fighters ($\text{fighter1_sub_acc} / \text{fighter2_sub_acc}$), we risk another divide-by-zero error.

Our Bayesian approach instead provides more nuanced, realistic estimates. For example:

- 10 attempts, 0 successes = 3.5% accuracy
- 9 attempts, 0 successes = 3.8% accuracy
- 8 attempts, 0 successes = 4.1% accuracy

This prevents over-punishing unsuccessful attempts while ensuring we never hit true zero. The accuracy gradually increases as sample size decreases, reflecting our increasing uncertainty with smaller sample sizes.

The V5 Redemption Arc

For V5, we're continuing to embrace AutoML (specifically AutoGluon) to eliminate the uncertainty in model optimization. Through V1-V3, I spent countless hours manually tuning gradient boosted algorithms like XGBoost and CatBoost. While I learned an enormous amount about hyperparameter optimization and various other tuning, I was never entirely confident I was meeting professional machine learning engineering standards.

AutoML removes that uncertainty. It systematically explores model architectures and hyperparameters in ways that would take me months to do manually. I still do significant tuning, but now it's faster and more reliable. No more wondering if I missed some crucial optimization or made rookie mistakes in the model architecture.

What This Means For Users

MMA-AI.net will continue hosting Model V4 predictions while V5 is under development. However, there won't be any improvements to the current model during this transition. If you want to take a step back and not ride with me for a month or two while I finish V5, I completely understand. This isn't about quick fixes - it's about building something that actually works.

The Bottom Line

Sometimes you need to lose six parlays in a row to light a fire under your ass and actually learn some math. But at least we're failing forward, and V5 will be built on more solid foundations.

P.S. To the UFC 309 fighters: those 3 AM tweets weren't personal. It was just the model and the Monster Energy talking.

Success! Then Failure?

March 20, 2024

I'm absolutely furious. UFC 309 just wiped out 6 units with 6 consecutive parlay losses. After four years of development, countless sleepless nights, and what I thought was a breakthrough in betting strategy, reality hit hard.

The Journey to Version 4

The evolution of MMA-AI.net has been a marathon. Version 1 took two years of meticulous development. When ChatGPT emerged, I saw an opportunity to rebuild everything from the ground up. Version 3 emerged after three months of intense work—I'm talking 5 AM bedtimes, night after night.

Then came Version 4, inspired by Chris from wolftickets.ai. His analysis revealed something fascinating: parlays could be more profitable than single picks when accounting for odds. This insight led to another complete overhaul, this time leveraging autoML to eliminate inefficiencies and using ChatGPT for guidance. The results were stunning: 50-60% ROI over six months.

Success Breeds Complacency

Those six months of success? They made me soft. I coasted. My initial \$270 investment had grown to \$13,000 purely through profit reinvestment—no additional capital needed. The model was working so well that I focused instead on rebuilding this website using Cursor, an AI-powered IDE. Despite knowing nothing about web development, HTML, or CSS, I managed to transform MMA-AI.net from an ad-riddled eyesore into what you see today.

Traffic surged to all-time highs. Everything seemed perfect. Then UFC 309 happened.

The Wake-Up Call

Six parlays. Six losses. 100% drawdown. The rage I'm feeling isn't just about the money—it's about the realization that I've been resting on my laurels. Those six months of coasting caught up with me in the most painful way possible.

This isn't the end of MMA-AI.net. Honestly, it's just what I needed to get my ass in gear and learn some math. I'm already planning the next evolution.

Stay tuned for my next post about where we go from here.

Welcome to the new MMA-AI.net

March 19, 2024

After three years of development, thousands of hours of feature engineering, and endless testing, I finally redid the stupid website. Goodbye ads, hello pretty new design. This platform represents what I believe to be one of the most sophisticated sports prediction models available today.

The past 5 months have been particularly exciting as we've cracked one of the final pieces of the profit-maximization puzzle: the betting strategy. Chris from wolftickets.ai and I knew there had to be a better approach than simply betting straight picks to maximize ROI. Chris was the first to discover it: parlays.

Since both of our models maintain significant accuracy and log loss advantages over Vegas, we can multiply that edge using parlays. Why? Because parlay odds aren't additive—they're multiplicative. Through extensive testing, we've found that 3-leg parlays offer the optimal balance of risk and reward. While 4-leg parlays actually showed higher ROI in testing, their boom-or-bust nature led to more extreme bankroll swings and higher bankruptcy risk. Since implementing the 3-leg strategy five months ago, the model has achieved a 50% ROI.

The Parlay Strategy

Our approach is straightforward: randomly selected AI picks combined into parlays, with no fighter appearing more than twice to prevent single-fighter dependency. The most common question I get is "Why not just parlay the +EV fighters together?" Well, I've tested hundreds of parlay permutations: underdogs only, favorites only, +EV only, 1-5 leg combinations—every variation imaginable—against a year of unseen fight data. Surprisingly, the +EV-only strategies consistently underperformed compared to randomly selected AI pick parlays.

This might seem counterintuitive, but it likely stems from how we're solving a binary classification problem. Our models excel at distinguishing wins (1) from losses (0), but may be less refined at setting precise win probabilities. I train on log loss, not accuracy—log loss being a metric that heavily penalizes confident mistakes while rewarding confident correct predictions. You can see evidence of this in the calibration curve on our About page.

But honestly? I don't care about the "why." Too many people get tunnel vision focusing on EV, which makes sense if you're working without a mathematical model like MMA-AI.net. But when you have a proven model with demonstrable advantages over Vegas odds, the only metric that matters is ROI. And our testing shows that random AI pick parlays consistently deliver the highest returns.

So here's to the new site, the new strategy, and the new model. You can find my predictions posted here on the home page and occasionally on [Twitter/X](#) or <https://reddit.com/r/mmabetting> before each event.



© 2024 MMA-AI.net